cuRAMSES I: Scalable AMR Optimizations for Large-Scale Cosmological Simulations

Juhan Kim[1]⋆
[1] Center for Advanced Computation, Korea Institute for Advanced Study, 85 Hoegiro, Dongdaemun-gu, Seoul 02455, Republic of Korea

Accepted XXX. Received YYY; in original form ZZZ

ABSTRACT

We present cuRAMSES, a suite of advanced domain decomposition strategies and algorithmic optimizations for the RAMSES adaptive mesh refinement (AMR) code, designed to overcome the communication, memory, and solver bottlenecks inherent in next-generation exascale cosmological simulations. The central innovation is a recursive k-section domain decomposition, which replaces the traditional Hilbert curve ordering with a hierarchical spatial partitioning. This approach substitutes global all-to-all communications with neighbour-only point-to-point communications. By maintaining a constant number of communication partners regardless of the total rank count, it significantly improves strong scaling at high concurrency. To address critical memory constraints at scale, we introduce a Morton-key hash table for octree-neighbour lookup alongside on-demand array allocation, drastically reducing the per-rank memory footprint. Furthermore, a novel spatial hash-binning algorithm in box-type local domains accelerates supernova and AGN feedback routines by over two orders of magnitude (a ∼260× speedup). For hybrid architectures, an automatic CPU/GPU dispatch model with GPU-resident mesh data reduces host–device transfer overhead to just 5.7% of the GPU runtime. Additionally, a variable-$N_{\rm cpu}$ restart capability enables flexible I/O workflows. Extensive diagnostics verify that all modifications preserve mass, momentum, and energy conservation to machine precision.

Key words: methods: numerical – cosmology: simulations – hydrodynamics – software: development

1 INTRODUCTION

Cosmological hydrodynamic simulations play a central role in modern astrophysics, connecting the predictions of the ΛCDM paradigm to the observable properties of galaxies, the intergalactic medium, and the large-scale structures of the Universe. Over the past decade, landmark galaxy formation simulations — IllustrisTNG (Pillepich et al. 2018; Nelson et al. 2019) with the moving-mesh code AREPO (Springel 2010), EAGLE (Schaye et al. 2015) with GADGET (Springel 2005), and the Horizon-AGN/Horzion-Run 5/NewHorizon/NewCluster simulations (Dubois et al. 2014; Lee et al. 2021; Dubois et al. 2021; Han et al. 2026) with the adaptive mesh refinement (AMR) code RAMSES (Teyssier 2002), among others (Hopkins et al. 2018; Davé et al. 2019; Tremmel et al. 2017) have advanced our understanding of cosmic structure.

These achievements, however, highlight the need for simulations of even greater scale and dynamic range. Capturing large-scale statistical properties with percent-level precision (Kim et al. 2011, 2015; Ishiyama et al. 2021; Maksimova et al. 2021; Schaye et al. 2015; Libeskind et al. 2018) requires volumes exceeding $(1\,h^{-1}\,\mathrm{Gpc})^3$ (**??**). At the same time, resolving the physics that governs individual galaxies including supermassive black hole (SMBH) feedback (Dubois et al. 2012; Weinberger et al. 2017), the multi-phase interstellar medium (ISM) (Springel & Hernquist 2003), and star formation on sub-parsec scales (Hopkins et al. 2014), demands deep AMR hierarchies with dynamic ranges of 10^5 or more. Achieving both goals simultaneously, large volume *and* high resolution requires an unprecedented computational effort that pushes well into the exascale regime (Habib et al. 2016; Frontiere et al. 2025). The principal barriers to reaching this regime are not only physical modelling limitations but also *algorithmic* challenges including communication overhead that grows with the number of `MPI` ranks, memory consumption that limits the refinement depth per node, and solver inefficiencies that dominate the wallclock time.

Among the numerical approaches employed by these simulations, AMR codes such as RAMSES and Enzo (Bryan et al. 2014) provide a particularly attractive framework, in which the computational mesh is refined only where the physics demands it, concentrating resources on collapsing haloes and star-forming regions while keeping the cost of smooth low-density regions manageable.

A key design choice in parallel AMR codes is the domain decomposition strategy for distributed-memory systems. Space-filling curves (SFCs), particularly the Hilbert (Peano–Hilbert) curve, have widely and successfully served this purpose (Warren & Salmon 1993; Springel 2005). The Hilbert curve maps the three-dimensional computational domain to a one-dimensional index preserving spatial local-

⋆ E-mail: kjhan@kias.re.kr

© 2026 The Authors

ity so that cells close in physical space remain close along the curve. For equal-volume partitioning, each rank's subdomain typically borders 6–8 neighbours in three dimensions. This enables a simple and effective partitioning where the one-dimensional index range is divided into $N_{\rm cpu}$ contiguous segments, each being assigned to an MPI rank. The resulting decomposition naturally produces compact subdomains with relatively small surface-to-volume ratios, minimising the ghost-zone boundary between neighbouring ranks.

However, scaling this approach to the regime of 10^{10}–10^{11} particles and 10^4–10^5 MPI ranks reveals fundamental limitations. The one-dimensional nature of the SFC means that *every* rank may, in principle, border *any other* rank in worst cases, forcing communication patterns that scale poorly with $N_{\rm cpu}$. In the standard RAMSES implementation, ghost-zone exchange, grid and particle migration, and sink particle synchronisation all rely on MPI_ALLTOALL to communicate counts among all ranks, leading to $\mathcal{O}(N_{\rm cpu}^2)$ communication complexity and $\mathcal{O}(N_{\rm cpu})$ per-rank buffer memory, which becomes a severe bottleneck when $N_{\rm cpu}$ exceeds $\sim 10^3$ (Teyssier 2002). Furthermore, the Hilbert load balancer assigns each cell a cost of $(80 + N_{\rm part})$, where $N_{\rm part}$ is the particle count per oct. While this does account for particles, the implied 80 times more weight to grid with respect to particle, severely underweights particle-dominated cells. For example, a grid hosting 10^4 particles in a dense halo carries a cost only $\sim 125\times$ that of an empty cell, whereas its actual memory footprint is about 450 times larger (taking $w_{\rm grid} = 2256\,{\rm B}$ for nvar=14 and $w_{\rm part} = 12\,{\rm B}$ per particle; see Section 2.3). The result is that particle-rich ranks are systematically overloaded in both memory and compute time (Springel 2005). The problem is particularly acute in cosmological zoom-in simulations (Lee et al. 2021; Dubois et al. 2021), where the high-resolution region occupies a small fraction of the total volume. The Hilbert curve concentrates nearly all refinement on a few ranks while the remaining ranks are left with low-resolution void cells, leading to severe load imbalance that worsens with increasing zoom factor. In the Horizon Run 5 simulation (Lee et al. 2021), for example, the Hilbert-based load rebalancing required more than 10 hours per invocation below redshift $z \approx 2$, and had to be abandoned entirely from the redshift.

Beyond communication and load balancing, per-rank memory overhead (e.g. the nbor and Hilbert key arrays, $>1\,$GB for $N_{\rm gridmax} = 5\,$M; Section 4) limits the refinement depth. The multigrid Poisson solver (Guillet & Teyssier 2011) dominates runtime ($\sim$50 per cent), and RAMSES' one-file-per-rank I/O prevents restarting with a different rank count.

The original RAMSES code relies exclusively on MPI for parallelism, assigning one MPI rank per processor core. To exploit the shared-memory bandwidth of modern multi-core nodes, hybrid MPI+OpenMP implementations have been developed. The Horizon Run 5 simulation (Lee et al. 2021) employs OMP-RAMSES, and the NewCluster zoom-in simulation (Han et al. 2026) employs RAMSES-yOMP, both adding OpenMP threading within each MPI rank for improved intra-node scalability. However, MPI+OpenMP alone still leaves the GPU compute capability of modern heterogeneous nodes untapped. With the emergence of exascale supercomputers whose floating-point throughput is dominated by GPU accelerators, a three (MPI+OpenMP+CUDA) level of parallelism is essential to fully utilise the cutting-edge of hardware available in modern technologies.

In this paper we describe CURAMSES, a comprehensive set of modifications to RAMSES that addresses each of these challenges. We introduce a recursive k-section domain decomposition (Section 2), adaptive MPI exchange backends (Section 3), a Morton key hash table (Section 4), multigrid and FFTW3 Poisson solver optimizations (Section 5), spatial hash binning for feedback (Section 6), variable-$N_{\rm cpu}$ restart (Section 7), and GPU-accelerated solvers (Section 8). Performance benchmarks are presented in Section 9, and we conclude in Section 10.

Throughout this paper, we use the notation of Teyssier (2002), where $N_{\rm levelmax}$ is the maximum AMR level, $N_{\rm gridmax}$ is the maximum number of grids per rank, and $N_{\rm cpu}$ is the total number of MPI ranks.

2 RECURSIVE K-SECTION DOMAIN DECOMPOSITION

We replace the Hilbert ordering with a recursive k-ary spatial partitioning that not only eliminates global MPI collectives but also encodes the communication pattern directly in the tree structure.

2.1 Hierarchical Partitioning of Spatial and Communication Domain

Given $N_{\rm cpu}$ MPI ranks, we first compute the prime factorization as

$$N_{\rm cpu} = p^{m_1} q^{m_2} \cdots r^{m_r}, \quad p > q > \cdots > r, \tag{1}$$

where one may note the order of factors. The splitting sequence is then,

$$\mathbf{k} = (\underbrace{p, \ldots, p}_{m_1}, \underbrace{q, \ldots, q}_{m_2}, \ldots, \underbrace{r, \ldots, r}_{m_r}), \tag{2}$$

which yields $L\,(= \sum_i m_i)$ levels in the tree structure encoding both the domain hierarchy and the communication pattern. The tree is a k-ary structure where each *node* represents a contiguous group of MPI ranks sharing a spatial sub-domain. The *root* node at level 0 spans all $N_{\rm cpu}$ ranks and each *leaf* node at level L corresponds to a single rank. At each level l, the domain of a node is split into k_l child nodes along the longest axis of the current bounding box. This longest-axis selection ensures roughly isotropic sub-domains minimising the surface-to-volume ratio and hence the ghost-zone count.

For example, $N_{\rm cpu} = 12 = 3 \cdot 2 \cdot 2$ produces the splitting sequence $(3, 2, 2)$ with $L = 3$ tree levels, where the root is split into 3 slabs along the longest axis, each slab is bisected, and each half is bisected again, yielding 12 leaf nodes, one per rank. Figure 1 illustrates this progressive decomposition for $N_{\rm cpu} = 12$, from the undivided domain (the upper left panel) through three successive levels of splitting with the corresponding k-section tree shown beneath each panel.

The tree is stored as a set of arrays indexed by node identifier. For each internal node, the child indices for each of the k_ℓ partitions and the spatial coordinates of the partition boundaries are recorded; for each leaf node, the assigned MPI rank is stored. Every node also carries the minimum and maximum rank indices of all leaves in its subtree, enabling rapid range queries during the hierarchical exchange.

The total number of tree nodes, $N_{\rm nodes} = 1 + \sum_{\ell=1}^{L} \prod_{i=1}^{\ell} k_i$,

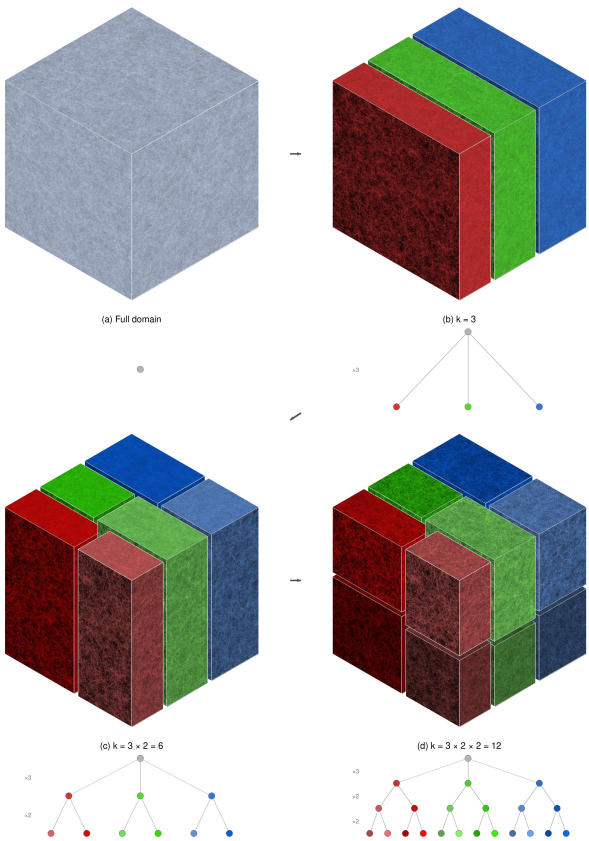

Figure 1. Progressive recursive k-section decomposition for $N_{\rm cpu} = 12 = 3 \times 2 \times 2$. (a) The full and undivided simulation domain. (b) First split into $k = 3$ slabs along the longest axis. (c) Each slab bisected along the second axis ($3 \times 2 = 6$ sub-domains). (d) Final bisection along the third axis ($3 \times 2 \times 2 = 12$ leaf domains). Below each panel, the corresponding k-section tree is shown. Sub-domain volumes vary by approximately 10–30 per cent, reflecting load-balanced wall placement.

is at most a few hundreds for practical values of $N_{\rm cpu}$ ($N_{\rm cpu} \leq 10^5$), representing negligible overhead.

2.2 Load-Balanced Wall Placement

When the tree is updated during load balancing (every `nremap` coarse steps), the wall positions at a certain level are adjusted by iterative binary search so that each partition receives a load proportional to the number of ranks it contains. The procedure operates level by level and top to bottom. At level ℓ, the following steps are performed.

(i) A histogram is built over the splitting coordinate. For each node at level ℓ, the cells within that node are projected onto the splitting axis and binned into a cumulative cost histogram with resolution $\Delta x_{\rm hist}$.

(ii) For each of the $k_\ell - 1$ walls within each node, a binary search adjusts the wall position until the cumulative load on the left side matches the target fraction. The target cumulative fraction for wall j in a node spanning ranks $[i_{\rm min}, i_{\rm max}]$ with total count $n = i_{\rm max} - i_{\rm min} + 1$ is

$$f_j = n^{-1} \sum_{m=1}^{j} n_m, \tag{3}$$

where n_m is the number of ranks assigned to partition m.

(iii) An `MPI_ALLREDUCE` aggregates the local histograms across all $N_{\rm cpu}$ ranks to obtain the global cumulative load at each wall position. The binary search converges when the relative load imbalance falls below a tolerance $\epsilon_{\rm tol}$ (typically 5 %), or when the wall position can no longer be resolved at the histogram resolution.

(iv) After wall convergence, the cells are repartitioned (sorted) according to the new wall positions and the histogram bounds are updated for the next level.

2.3 Memory-Weighted Cost Function

The default RAMSES load balancer uses a cost of $80 + N_{\rm part}$ per oct grid, which underweights particle-dominated cells by a factor of ∼3–4 relative to their true memory footprint. CU-RAMSES replaces this with a memory-weighted cost function

$$C_{\rm cell} = \frac{w_{\rm grid} + n_{\rm part}(\texttt{igrid}) \cdot w_{\rm part} + n_{\rm sink}(\texttt{igrid}) \cdot w_{\rm sink}}{2^{N_{\rm dim}}}, \tag{4}$$

where $w_{\rm grid}$ is the memory cost per grid slot, computed automatically as

$$w_{\rm grid} = 2^{N_{\rm dim}} \times (2\,\texttt{nvar} \times 8 + 52) + 48 \tag{5}$$

in bytes, for example, 1232 B(ytes) for `nvar`=6 or 2256 B for `nvar`=14, accounting for hydro (`uold`+`unew`), gravity, and AMR bookkeeping arrays. A user-specified value may override the automatic formula. And $w_{\rm part}$ is the memory cost per particle slot (default 12 bytes for position, velocity, mass, and linked-list pointers), $n_{\rm part}(\texttt{igrid})$ is the number of dark matter and star particles (both sharing the same memory layout) attached to grid, `igrid`, and $w_{\rm sink}$ is a computational weight per sink particle (default 500) that accounts for the expensive feedback and accretion operations associated with each sink. The sink count $n_{\rm sink}(\texttt{igrid})$ is obtained by descending the AMR tree from each sink position to its leaf cell. The division by $2^{N_{\rm dim}}$ distributes the grid cost evenly among its eight cells.

This cost function ensures that ranks hosting dense haloes (many particles per cell) receive fewer cells, preventing memory exhaustion on particle-heavy ranks. Our Cosmo256 test (for details, see Section 9.1; 200 M particles, 12 ranks) shows that memory-weighted balancing reduces the peak-to-mean memory ratio from 2.5 to 1.3, and keeps the per-rank memory imbalance $M_{\rm max}/M_{\rm min} \leq 1.05$ across 2–64 ranks (Section 9.5), without affecting physics results such as identical numbers of new stars, sinks, $e_{\rm cons}$ (fractional total energy conservation error), $e_{\rm pot}$ (gravitational potential energy), and $e_{\rm kin}$ (kinetic energy) to machine precision after advancing ten coarse time steps.

3 AUTO-TUNING MPI COMMUNICATIONS

Ghost-zone (virtual boundary) exchange is the most communication-intensive operation in RAMSES, invoked mul-

tiple times per fine time step for hydro variables, gravitational potential, particle flags, and density contributions. Each exchange involves the same algorithmic pattern (pack data from local grids, transfer to remote ranks, and unpack into ghost zones), but the underlying MPI transport can be realised in fundamentally different ways. cuRAMSES implements two additional methods and adaptively selects one of the three communication backends (MPI_ALLTOALL/P2P/K-Section) at runtime.

3.1 Exchange Methods

3.1.1 MPI_ALLTOALLV

The MPI_ALLTOALLV collective exchanges variable-length messages among all ranks in a single call. This backend packs all emission data (data to be sent to other ranks) into a contiguous buffer with displacements computed from per-rank send counts, invokes MPI_ALLTOALLV, and unpacks the received data. Although it involves all $N_{\rm cpu}$ ranks (with zero counts for non-neighbours), "high-quality" MPI implementations exploit the sparsity internally and can outperform explicit P2P on certain interconnects, particularly for small messages on fat-tree topologies. However, the implicit global synchronisation and $\mathcal{O}(N_{\rm cpu})$ memory overhead make it less attractive at very large rank counts.

3.1.2 Point-to-point (P2P)

The P2P backend follows the original RAMSES pattern of non-blocking sends/receives (MPI_ISEND/MPI_IRECV) with MPI_WAITALL. The number of communication partners is set by the domain geometry (typically 6–8), independent of $N_{\rm cpu}$, making P2P the best choice when neighbour sets are sparse.

3.1.3 Hierarchical K-Section Exchange

The k-section backend replaces the global communication pattern with a hierarchical tree walk using the domain decomposition tree described in Section 2.1. The algorithm walks the tree from root to leaf, where each node represents a contiguous group of MPI ranks. At each level ℓ, a node is partitioned into k_ℓ child nodes and each rank communicates with at most $k_\ell - 1$ correspondents (one per sibling subtree). This spreads the communication load evenly across the sibling rather than funnelling all traffic to a single rank.

The total number of communications per rank per exchange is

$$N_{\rm com} = \sum_{\ell=1}^{L}(k_\ell - 1) = \sum_i m_i(p_i - 1), \tag{6}$$

which for $N_{\rm cpu} = 1024 = 2^{10}$ gives $N_{\rm com} = 10$, which is two orders of magnitude fewer than the ∼1024 communications in an all-to-all pattern. Even in the best case, where the MPI library internally optimises MPI_ALLTOALLV to skip zero-length communications, a rank with six neighbours must still perform at least six pairwise transfers (and often more on non-ideal interconnects), so the k-section communication count of $\mathcal{O}(\sum_l k_l)$ remains competitive. Crucially, $N_{\rm com}$ depends only on the prime factorization of $N_{\rm cpu}$ and is completely independent of the degree of particle clustering or the AMR refinement level, so the communication topology remains fixed regardless of how complex the simulation becomes. Figure 2 illustrates the communication pattern for $N_{\rm cpu} = 12$. Each rank sends $N_{\rm com} = (3-1) + (2-1) + (2-1) = 4$ messages per exchange, independent of $N_{\rm cpu}$.

Table 1. Communication complexity per exchange operation. $N_{\rm nb}$ is the number of neighbour ranks, $N_{\rm gh}$ the number of ghost grids per rank, k_l the branching factor at tree level l, and $k_{\rm max} = \max(k_l)$.

	P2P	ALLTOALLV	k-section
Msgs	$\mathcal{O}(N_{\rm nb})$	$\mathcal{O}(N_{\rm cpu})$	$\mathcal{O}(\sum_l k_l)$
Buffer	$\mathcal{O}(N_{\rm nb}N_{\rm gh})$	$\mathcal{O}(N_{\rm cpu}N_{\rm gh})$	$\mathcal{O}(k_{\rm max}N_{\rm gh})$
Sync.	none	barrier	none

The pairing structure at each tree level follows directly from the branching factor. When a node has p children, data from every child must reach every other child, accomplished in $p-1$ sequential steps. In step s ($s = 1, \ldots, p-1$), child $s+1$ exchanges with each of the children $1, \ldots, s$, yielding s concurrent point-to-point communications. After all $p-1$ steps, every child possesses the complete data set of the entire subtree.

A trade-off is that the total data volume per rank exceeds that of direct P2P or MPI_ALLTOALLV since items may be forwarded through multiple tree levels. In the worst case, a rank relays its entire buffer at every level, giving at most $L \times V_{\rm local}$, where $V_{\rm local}$ is the local data volume. In practice, items are filtered by child index at each level and successive levels operate on progressively smaller subsets. The key advantage, namely reducing the number of communication partners from $\mathcal{O}(N_{\rm cpu})$ to $\mathcal{O}(\sum_\ell k_\ell)$, more than compensates for the modest volume increase particularly at large $N_{\rm cpu}$, where the startup latency in the network hand-shaking dominates.

Table 1 compares the communication complexity of the three backends. The k-section exchange also extends to ghost-zone operations (forward, reverse, integer, and bulk variants), communication structure construction (replacing the original MPI_ALLTOALL-based build_comm), and feedback collectives, eliminating all global communication patterns from the AMR infrastructure.

3.2 Adaptive Method Selection

No single backend dominates in performance across all exchange types, rank counts, and simulation stages. During the initial mesh setup and I/O phases, communication is sparse and P2P excels. After load rebalancing, the neighbour topology changes and a previously inferior method may become optimal. As the simulation evolves and AMR levels deepen, ghost-zone volumes grow and the relative cost of each backend shifts. cuRAMSES therefore implements a continuous per-component auto-tuning framework that adapts to the evolving communication pattern throughout the simulation.

3.2.1 Identification of exchange components

Seven independent exchange components are identified, each with its own data size and call frequency. (i) fine_dp, for-

Figure 2. Hierarchical exchange communication pattern for $N_{\rm cpu} = 12\,(= 3 \times 2 \times 2)$. Coloured rectangles represent `MPI` ranks numbered 1–12, using the same colour scheme as Figure 1. Arcs denote bidirectional point-to-point exchanges. At level 1 ($k_1 = 3$), two steps connect each rank with correspondents in the two sibling subtrees; the two colours in step 2 distinguish the children 1↔3 (dark red) and children 2↔3 (orange) pairings that proceed concurrently. At levels 2 and 3 ($k_2 = k_3 = 2$), the communication range contracts to within each group and then to adjacent pairs. The total communication count per rank is $N_{\rm com} = (3-1) + (2-1) + (2-1) = 4$.

ward double-precision ghost exchange (hydro, potential). (ii) `fine_int`, forward integer ghost exchange (`cpu_map`, flags). (iii) `reverse_dp`, reverse accumulation (density). (iv) `reverse_int`, reverse integer accumulation. (v) `pair_int`, paired integer exchange. (vi) `bulk_dp`, bulk forward exchange ($N_{\rm var}$ variables in one call). (vii) `bulk_rev_dp`, bulk reverse accumulation. Each component maintains its own auto-tune state, so a method that is optimal for small integer exchanges need not be the same as for large bulk hydro transfers.

3.2.2 Auto-tune protocol

Each component independently progresses through four phases. (0) trial P2P, (1) trial k-section, (2) production with the faster backend, and (3) periodic probing of the alternative. Phase 2 tracks performance via an exponential moving average (EMA, $\alpha = 0.05$) as

$$\bar{t}_{n+1} = \alpha t_n + (1-\alpha)\bar{t}_n, \tag{7}$$

where α is the update rate, t_n is the elapsed time of the current call and $\bar{t}_n$ is the running EMA from the previous step. When the probe in Phase 3 finds the alternative faster by >20%, the component switches.

4 MORTON KEY HASH TABLE AND MEMORY OPTIMIZATIONS

4.1 The Neighbour Array (`nbor`) Problem

RAMSES stores the octree connectivity in several arrays, the largest of which is `nbor(1:ngridmax, 1:`$2^{\rm dim}$`)`, a six-column integer array (in 3D) that records, for each grid, the cell index of its neighbour in each of the six Cartesian directions ($\pm x$, $\pm y$, $\pm z$). Each entry is a 64-bit integer occupying 8 bytes, so this array consumes 48 $N_{\rm gridmax}$ bytes (240 MB for $N_{\rm gridmax} = 5$ M). Moreover, the `nbor` array must be maintained during grid creation, deletion, defragmentation, and inter-rank migration, which is a significant source of code complexity and a potential source of bugs.

4.2 Morton Key Encoding and Neighbour Arithmetic

A Morton key (also known as a Z-order key) (Morton 1966; Samet 2006) is an integer formed by interleaving the bits of the three-dimensional integer coordinates (i, j, k) of a grid at its AMR level as

$$M(i,j,k) = \sum_{b=0}^{B-1} \Big[\mathrm{bit}_b(i)\cdot 2^{3b} + \mathrm{bit}_b(j)\cdot 2^{3b+1} + \mathrm{bit}_b(k)\cdot 2^{3b+2}\Big], \tag{8}$$

where B is the number of bits per coordinate and $\mathrm{bit}_b(n)$ extracts bit b of integer n. Two key widths are supported via a compile-time flag: a 64-bit key with $B = 21$ (default, compatible with the Intel `ifx` compiler), and a 128-bit key with $B = 42$. At AMR level ℓ, the integer coordinate range is $[0,\, 2^{\ell-1} n_x)$, where n_x is the number of root-level cells per dimension. With $B = 21$ the maximum allowed level is 22 ($n_x = 1$) or 20 ($n_x = 4$) while $B = 42$ extends these to 43 and 41, respectively.

The integer coordinates of a grid at level ℓ are computed from its floating-point centre position $\mathbf{r}_g$ as

$$i_d = \texttt{iFloor}(2^{\ell-1} r_{g,d}), \quad d \in \{x, y, z\}, \tag{9}$$

where `iFloor` is the integer floor function and coordinates are in units of the coarse grid spacing. The integer coordinates use zero-based (C-style) indexing, starting from $i_d = 0$ even though Fortran arrays are conventionally one-based. Note that AMR does not populate all possible grid positions at a given level since only regions that satisfy the refinement criteria contain grids. The Morton key therefore serves as a unique spatial address for each *existing* grid. The hash table (Appendix B) stores only the grids that are actually allocated making the look-up cost independent of the total number of potential grid positions at that level.

The neighbour key in direction j is obtained by decoding, shifting the appropriate coordinate with periodic wrapping, and re-encoding (Appendix A). Parent and child keys follow from 3-bit shifts as $M_{\rm parent} = M{>>}3$, $M_{\rm child} = (M{<<}3)\,|\,i_{\rm child}$.

The Morton keys are stored in a per-level open-addressing hash table that maps keys to grid indices, providing $\mathcal{O}(1)$

expected neighbour look-up (Appendix B). Each hash table entry stores a 16-byte Morton key and a 4-byte grid index (20 bytes total). With a maximum filling factor of 1.4 (i.e. 70 per cent occupancy), the aggregate memory footprint across all levels is approximately $1.4 \times 20 \times N_{\rm grids} = 28\,N_{\rm grids}$ bytes, where $N_{\rm grids}$ is the number of allocated grids. For a typical run with $N_{\rm grids} \approx 3\,$M, this amounts to roughly 83 MB, far smaller than the original `nbor` array ($48\,N_{\rm gridmax} \approx 240\,$MB for $N_{\rm gridmax} = 5\,$M).

The hash table memory cost is far smaller than the original `nbor` cost of $48\,N_{\rm gridmax}$ bytes. Combined with Hilbert key elimination and on-demand allocation of auxiliary arrays, the total savings exceed 1.2 GB per rank (Appendix C).

5 POISSON SOLVER OPTIMIZATIONS

Solving the Poisson equation $\nabla^2\phi = 4\pi G\rho$ for self-gravity is one of the most expensive operations in cosmological AMR simulations. In baseline RAMSES, performance profiling reveals that the Poisson solver accounts for approximately half of the total wallclock time per coarse step. And this fraction grows further as the AMR structure develops and deeper levels are populated. We describe two complementary strategies, optimizing the iterative multigrid solver that operates on all AMR levels (Section 5.1), and replacing it with a direct FFT solve on the uniform base level (Section 5.2). Together these reduce the Poisson fraction to approximately 40%.

5.1 Multi-Grid Method

RAMSES employs a multigrid V-cycle (Brandt 1977; Briggs et al. 2000; Trottenberg et al. 2001) with red-black Gauss–Seidel (RBGS) smoothing (Wesseling 1992; Adams 2001) at each AMR level.

5.1.1 V-Cycle Algorithm and Exchange Optimization

Starting at the finest AMR level, the algorithm applies RBGS pre-smoothing and then *restricts* the residual to the next coarser level. Restriction requires no ghost exchange because each coarse cell is the average of its eight fine children, all of which reside on the same rank by the oct-tree construction. This restriction–smooth sequence descends to the coarsest level, where a direct solve (or additional smoothing) is applied. The correction is then *prolongated* (interpolated) back up through each level, with post-smoothing at each step in order to damp high-frequency errors introduced by the interpolation. Unlike restriction, prolongation *does* require a ghost exchange. Specifically the trilinear (3D) interpolation stencil reads neighbouring coarse cells which may belong to a different rank. Figure 3 illustrates the V-cycle structure and the exchange positions.

In a distributed-memory implementation each smoothing step requires ghost-zone exchanges between `MPI` ranks to update boundary values. In the original code, every red sweep, black sweep, residual computation, and norm reduction each triggers a separate communication (ghost exchange or `MPI_ALLREDUCE`), totalling 9 communications per smoothing cycle per level (4 for pre-smoothing, 4 for post-smoothing, plus 1 for prolongation).

We apply several targeted optimizations as follows.

(i) **Precomputed neighbour cache.** Neighbour grid indices are precomputed into a contiguous array before the V-cycle, reducing the cost of hash table lookups over all iterations (see Appendix D).

(ii) **Merged red-black sweep.** The inter-sweep ghost exchange between the red and black passes is removed. The black sweep proceeds using boundary values that have not yet been refreshed from neighbouring ranks. Because the multigrid smoother is used only as a preconditioner (not as a standalone solver), these slightly *outdated* boundary values do not affect convergence since any small error they introduce will be corrected by the next iteration. This means that we need not be bothered by the fresh communication of data from neighbours. This saves a number of ghost communications accordingly.

(iii) **Fused residual and norm.** The residual $r_i \equiv f_i - (\nabla_h^2\phi)_i$, where $f_i \equiv 4\pi G\rho_i$ is the source term and ∇_h^2 is the discrete Laplacian, and its squared norm $\sum_i r_i^2$ are computed in a single loop, eliminating one exchange per iteration.

These reduce the exchange count from 9 to 5 per iteration, as shown in the lower panel of Figure 3. The merged red-black technique is a standard relaxed-synchronisation strategy for distributed multigrid solvers (Adams 2001; Briggs et al. 2000). Implementation details are given in Appendix D.

5.1.2 Performance Impact

Combining all optimizations, the MG Poisson solver's share of total runtime is reduced from approximately 50 per cent to 40 per cent in a Cosmo256 test (200 M particles, 12 ranks, 10 coarse steps). The iteration counts are unchanged (Level 8, 5 iterations and Level 9, 4 iterations), confirming that the merged red-black exchange does not degrade convergence.

5.2 FFTW3 CPU Direct Poisson Solver

As a complement to the iterative optimizations described above, we replace the MG V-cycle on fully uniform AMR levels with a single direct FFT solve. At the base level (`levelmin`), all $N = N_x N_y N_z$ cells exist and the grid is periodic making the Poisson equation $\nabla^2\phi = 4\pi G\rho$ solvable exactly via spectral methods. A further advantage of the Fourier-space representation is that modified gravity models (e.g. massive neutrinos, coupled dark energy perturbations) can be incorporated by multiplying the Green's function with a scale- and time-dependent transfer function without altering the real-space solver infrastructure.

The solver uses FFTW3 (Frigo & Johnson 2005) and operates in two regimes depending on the grid size.

5.2.1 Small grid ($N \leq 256^3$)

Each rank gathers its local right-hand side into a local array and an `MPI_ALLREDUCE` sums the contributions. A local in-place R2C FFT (multi-threaded via FFTW3+OpenMP) solves the system.

5.2.2 Large grid ($N > 256^3$)

The global grid is distributed across ranks using FFTW's native slab decomposition along the x-axis. Data redistribu-

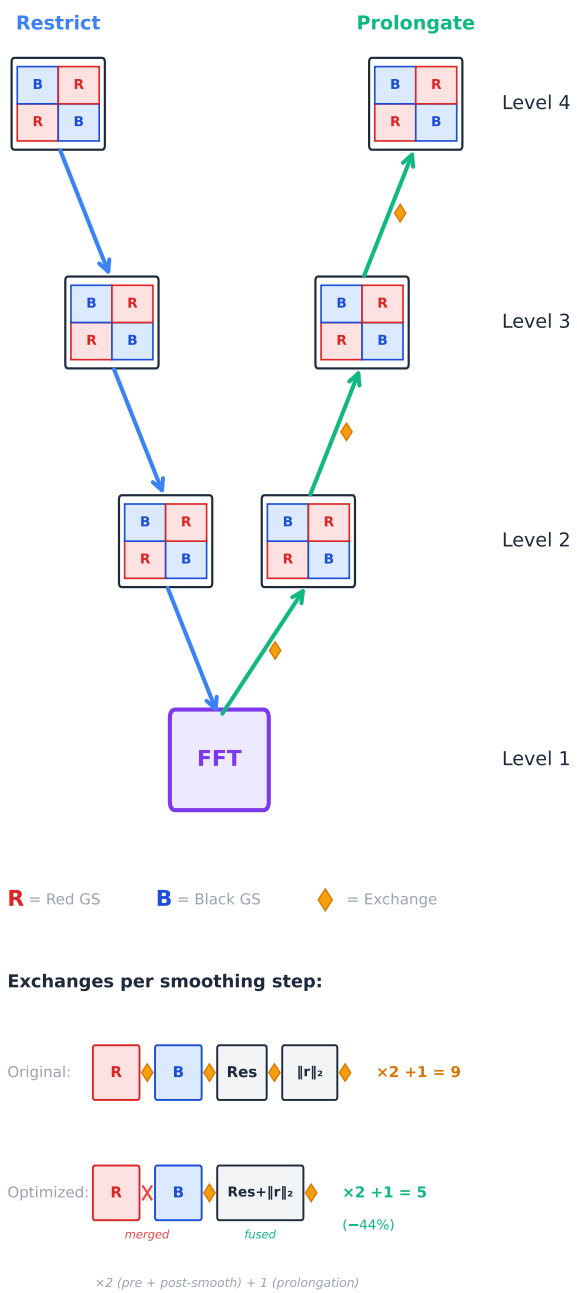

Figure 3. Schematic of the multigrid V-cycle for four AMR levels. *Upper:* The left (descending) leg restricts the residual from fine to coarse; the right (ascending) leg prolongates the correction. Orange diamonds on the prolongation arrows mark ghost-zone exchanges. *Lower:* Exchange positions within one smoothing step. The original code requires 4 exchanges (after red, after black, residual, norm). However, the optimized code merges the red-black sweep and fuses the residual and norm computation, reducing the count to 2. With two smoothing steps (pre + post) plus one prolongation exchange, the total number decreases from 9 to 5 per level per iteration.

tion between RAMSES's spatial domain decomposition and FFTW's slab layout requires an all-to-all exchange in both forward and reverse directions.

5.2.3 Sparse point-to-point exchange

We replace `MPI_ALLTOALLV` with sparse `MPI_ISEND`/`MPI_IRECV` targeting only actual communication partners, since each rank's domain overlaps only a small number of FFTW slabs. Partner lists are precomputed from the spatial bounding boxes and cached until the domain decomposition changes, reducing the communication count from $O(N_{\rm cpu})$ to $O(n_{\rm partners})$.

5.2.4 Performance

The FFTW3 solver reduces the base-level Poisson time from 241 s (MG V-cycle) to 28.9 s (**8.3× speedup**) on 12 ranks with a 512^3 base grid, matching the cuFFT GPU result (28 s) without requiring a GPU. The solver is enabled via `use_fftw=.true.` in `&RUN_PARAMS` and requires compilation with `make USE_FFTW=1`.

Partner lists are computed from bounding boxes via `MPI_ALLGATHER`, so the solver works identically with both Hilbert and k-section orderings.

6 FEEDBACK SPATIAL BINNING

Feedback routines must identify all cells within a finite blast radius of each event. We introduce spatial hash binning to accelerate this search, reducing it from an $\mathcal{O}(N_{\rm cells}\times N)$ pairwise scan to an $\mathcal{O}(N_{\rm cells})$ local lookup. With k-section ordering, the binning grid covers only the local domain bounding box (plus an $r_{\rm max}$ halo), avoiding global memory allocation and naturally restricting neighbouring interactions to a compact region.

6.1 The Brute-Force Bottleneck

The Type II supernova (SNII) feedback implementation in RAMSES involves two computationally expensive routines.

- `average_SN`: averages hydrodynamic quantities within the blast radius of each SN event, accumulating volume, momentum, kinetic energy, mass loading, and metal loading. The original implementation loops over all cells with respect to all SNe, yielding $\mathcal{O}(N_{\rm cells}\times N_{\rm SN})$ complexity.
- `Sedov_blast`: injects the blast energy and ejecta into cells within the blast radius. Same $\mathcal{O}(N_{\rm cells}\times N_{\rm SN})$ complexity.

In production simulations with ∼2000 simultaneous SN events, these routines consume 66 s and 11 s per call respectively, dominating the feedback time step.

6.2 Spatial Hash Binning

We partition the simulation domain into a uniform grid of $n_{\rm bin}^3$ bins, where

$$n_{\rm bin} = \texttt{max}\big[1,\,\texttt{min}\big(128,\,\texttt{iFloor}(L_{\rm box}\,/\,r_{\rm max})\big)\big] \tag{10}$$

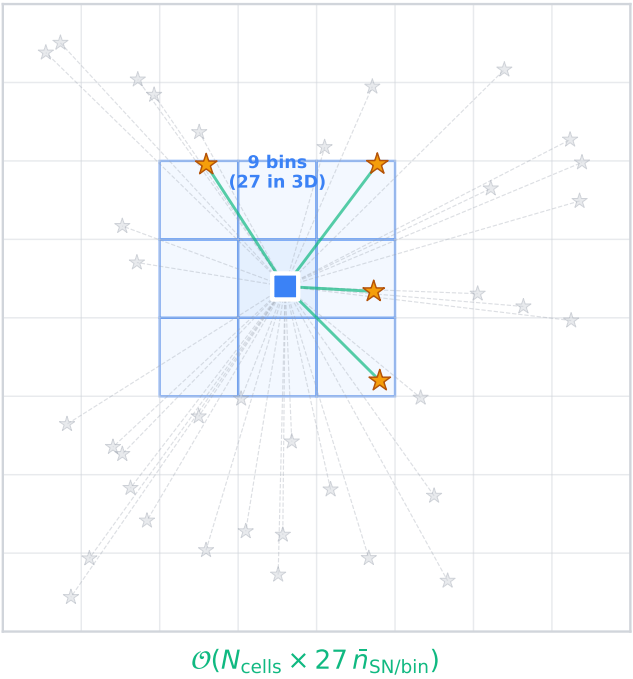

Figure 4. Gathering scheme on the spatial hash binning for feedback (shown in 2D for clarity). The domain is partitioned into a uniform grid of bins. The target cell (blue square) checks only the 9 neighbouring bins (27 in 3D), shown as shaded regions. Solid green lines connect the cell to SN events (orange stars) within the neighbourhood. Grey dashed lines indicate distant SN events that are skipped, reducing the complexity from $\mathcal{O}(N_{\rm cells} \times N_{\rm SN})$ to $\mathcal{O}(N_{\rm cells} \times 27\,\bar{n}_{\rm SN/bin})$.

and $r_{\rm max}$ is the maximum SN blast radius (the larger of `rcell` × `dx_min` and `rbubble`). Each SN event is assigned to a bin based on its position, and a linked list threads the events within each bin.

For each cell, we compute its bin index and check only the 27 neighbouring bins in three dimensions. Since $r_{\rm max}$ is at most the bin size by construction, this 27-bin neighbourhood is guaranteed to contain all SNe that could influence the cell. The complexity becomes $\mathcal{O}(N_{\rm cells} \times \bar{n}_{\rm SN/bin} \times 27)$, where $\bar{n}_{\rm SN/bin} = N_{\rm SN}/n_{\rm bin}^3$ is the average number of SNe per bin. Figure 4 illustrates this approach.

6.3 Performance Results

Both routines are parallelised with OpenMP over grids using a *gathering* scheme: the outer loop iterates over cells (distributed across threads by grid ownership), and each cell looks up nearby SN events from the spatial bins. This gathering approach is well-suited to OpenMP because each thread processes its own set of grids. The alternative *scattering* approach—iterating over SN events and modifying all affected cells—would incur race conditions whenever multiple events overlap, requiring expensive locking. In the gathering scheme, `Sedov_blast` writes only to locally owned cells (one thread per grid) and needs no synchronisation, while `average_SN` accumulates contributions into shared per-SN arrays using lightweight `!$omp atomic` updates.

Measured on the Cosmo512 test (Section 9.1; 12 `MPI` ranks, k-section ordering) at the restart epoch with approximately 2000 simultaneous SN events, the combined gains from spatial binning and OpenMP gathering are as follows.

- `average_SN`: 66 s → 0.25 s (∼260× speedup)
- `Sedov_blast`: 11 s → 0.07 s (∼157× speedup)

The same spatial binning technique is applied to the AGN feedback routines (`average_AGN` and `AGN_blast`), which suffer from the same $\mathcal{O}(N_{\rm cells} \times N_{\rm AGN})$ brute-force scaling. In production simulations with tens of thousands of active AGN sink particles, these routines dominate the sink-particle time step.

The AGN feedback involves three distinct interaction modes (saved energy injection, jet feedback, and thermal feedback), each with a different geometric distance criterion. The spatial binning is agnostic to these distinctions and simply reduces the candidate AGN set from the full population to only those in the 27 neighbouring bins, while preserving all distance-check logic and physical calculations unchanged. The linked-list construction and 27-bin traversal follow the same pattern as the SNII implementation (§6.2), with `bin_head` and `agn_next` arrays replacing the SN-specific versions.

With approximately 32 000 active AGN particles, the binned `average_AGN` achieves a 30× speedup and `AGN_blast` a 14× speedup, reducing the total AGN feedback time by a factor of ∼4. Verification confirms bit-identical conservation diagnostics compared to the original brute-force implementation.

6.4 Sink Particle Merger: Oct-Tree FoF with Auto-Tuning

Sink particles in RAMSES represent unresolved compact objects (black holes, proto-stars) and must be merged when they approach within a linking length $r_{\rm link} = r_{\rm merge}\,\Delta x_{\rm min}$. The standard merge routine uses a Friends-of-Friends (FoF) algorithm with $\mathcal{O}(N_{\rm sink}^2)$ brute-force pair enumeration, in which for each connected component (group) it swaps particles into a contiguous block and iterates until no new links are found. Although $N_{\rm sink}$ is typically small ($\lesssim 10^3$), production simulations of galaxy clusters can reach $N_{\rm sink} \sim 10^5$ (e.g. Horizon Run 5; Lee et al. 2021), at which point the $\mathcal{O}(N^2)$ scaling becomes prohibitive.

We provide three FoF backends with automatic selection.

6.4.0.1 Sequential brute-force. The original $\mathcal{O}(N^2)$ algorithm, preserved as a correctness reference. But note that the order of sink array changes by swapping a pair of sinks during the FoF link.

6.4.0.2 Serial oct-tree. An oct-tree is built over the sink positions. For each sink particle, tree nodes whose bounding sphere does not overlap the linking sphere are pruned, reducing the average neighbour search to $\mathcal{O}(N \log N)$. Linked particles are erased from the tree so subsequent searches skip them. This is equivalent to the brute-force result but dramatically faster for $N_{\rm sink} \gtrsim 500$.

6.4.0.3 OpenMP oct-tree. Every `MPI` rank builds the same oct-tree from the full (replicated) sink particle list, so no

MPI data distribution is involved that the parallelism is purely intra-rank via OpenMP. The particle index range is divided among OpenMP threads, and all threads share the same read-only tree but maintain private inclusion and group-ID arrays. A thread-safe variant of FoF_link (Omp_FoF_link) replaces the sequential tree-erasure step with a thread-local included array, enabling concurrent traversal without locks.

Cross-boundary groups are rolled back to the owning thread, ensuring deterministic results at negligible cost ($<$ 0.1 per cent of particles form multi-member groups). Thread-local results are consolidated after the parallel phase and the final result is identical on all ranks.

6.4.0.4 Auto-tuning. An Auto_Do_FoF wrapper probes all three backends during the first call, measures their wall-clock time, and selects the fastest via MPI_BCAST consensus. Subsequent calls bypass the probe and invoke the winner directly.

All three methods produce identical group partitions for every test case ($N_{\rm sink}$ from 1 to 10^5), verified by an $\mathcal{O}(N)$ bidirectional group-mapping check. At $N_{\rm sink} = 10^5$ the OpenMP oct-tree is about 600 times faster than brute-force (Figure 5).

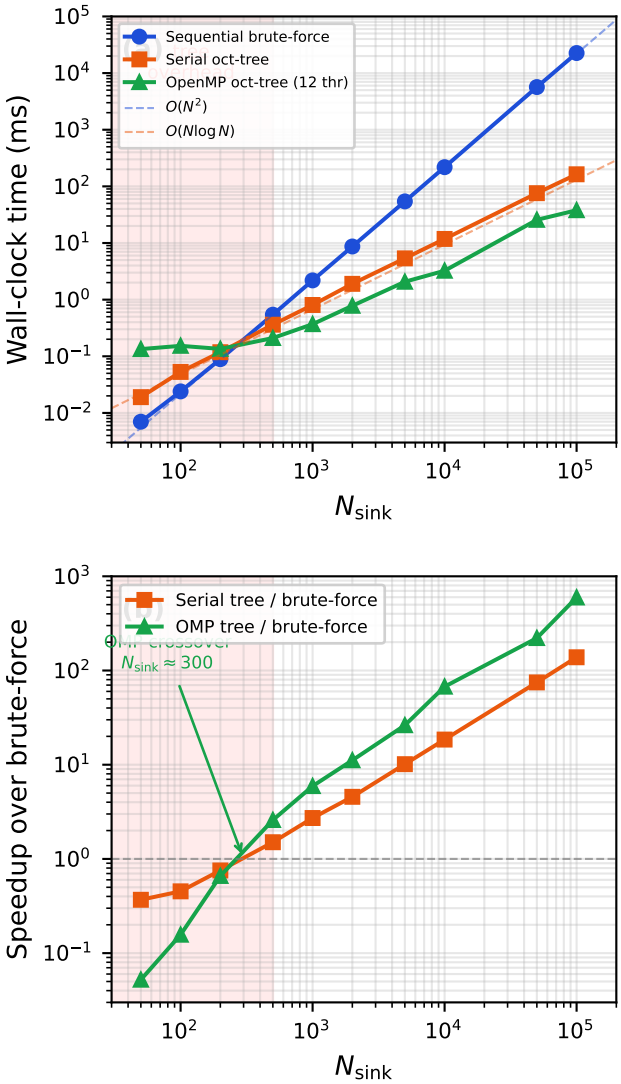

Figure 5. FoF merger wallclock time as a function of the number of sink particles for the three backends. We randomly distribute the test sink particles. The sequential brute-force (blue) scales as $\mathcal{O}(N^2)$, the serial oct-tree (orange) scales as $\mathcal{O}(N \log N)$, and the OpenMP oct-tree (green, 12 threads) achieves a further parallel speedup of about 4.3. The test uses a massively populated, highly clustered particle distribution in which nearly every particle has close neighbours, representing a worst-case scenario for the FoF algorithm. The dashed line shows the $\mathcal{O}(N^2)$ scaling for reference. At $N_{\rm sink} = 10^5$, the parallel tree is about 600 times faster than brute-force.

7 VARIABLE-NCPU RESTART

7.1 HDF5 Parallel I/O and Restart

Standard RAMSES writes one binary file per MPI rank per output. Restarting with a different number of ranks is not directly supported, requiring an intermediate step of reading with the original rank count, redistributing, and re-writing.

We implement HDF5 parallel I/O using the HDF5 library's MPI-IO backend. All ranks write to and read from a single HDF5 file, with datasets organized hierarchically. Figure 6 illustrates this hierarchy.

When the number of ranks in the output file ($N_{\rm file}$) differs from the current run (when $N_{\rm file} \neq N_{\rm cpu}$), the following procedure executes during restart.

(i) Build a uniform k-section tree for the new $N_{\rm cpu}$ (equal-volume partitioning, without load-balance adjustment).
(ii) Read all grids from the HDF5 file. Since the file is a single shared file, all ranks can access all data.
(iii) For each grid, compute the CPU ownership from the father cell's position.
(iv) Each rank retains only the grids assigned to it, building the local AMR tree incrementally.
(v) Hydro, gravity, and particle data are read and scattered to locally owned grids using a precomputed file-index-to-local-grid mapping.
(vi) On the first coarse step after restart, a forced load-balance operation redistributes grids for optimal balance under the new rank configuration.

This approach requires that all ranks temporarily hold the full grid metadata (positions and son flags) during the reconstruction phase. For typical production outputs ($\sim$10 M total grids), this temporary overhead is a few hundred MB, well within the memory budget freed by the optimizations of Section 4.

7.2 Restarting Issues

When HDF5 is unavailable or the native binary format (informat='origin') is preferred, a distributed I/O strategy enables variable-$N_{\rm cpu}$ restart from the per-rank binary files written by standard RAMSES. The binary format stores one file per MPI rank (amr_XXXXX.outYYYYY, hydro_XXXXX.outYYYYY, etc.) so that the number of files equals $N_{\rm file}$, which may differ from the current $N_{\rm cpu}$.

The restart proceeds in three stages as follows. First, files are distributed among ranks by round-robin assignment. Sec-

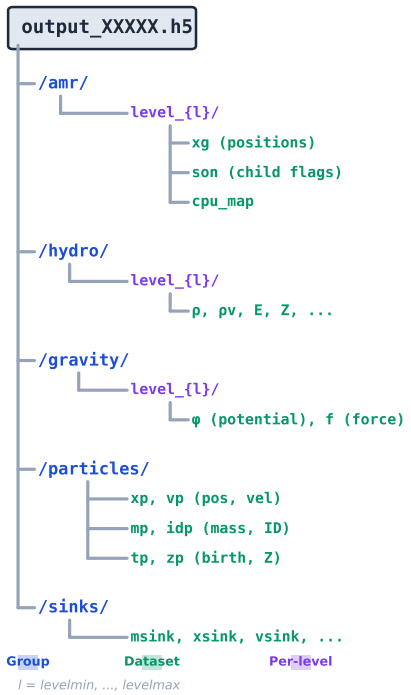

Figure 6. HDF5 output file hierarchy. AMR, hydro, and gravity data are stored per level while particles and sinks are stored as flat arrays. All ranks write to a single shared file via `MPI`IO.

Table 2. Restart format compatibility. "Any" means the output can be restarted with an arbitrary number of `MPI` ranks; "Same" requires $N_{\rm cpu} = N_{\rm file}$; "Diff" requires $N_{\rm cpu} \neq N_{\rm file}$ (the variable-$N_{\rm cpu}$ path rebuilds the decomposition from scratch).

Format	File DD	Run DD	$N_{\rm cpu}$	OK?
Binary	Hilbert	Hilbert	Any	Yes
Binary	Hilbert	k-section	Diff	Yes
Binary	k-section	k-section	Any	Yes
Binary	k-section	Hilbert	Diff	Yes
HDF5	Hilbert	Hilbert	Same	Yes
HDF5	Hilbert	k-section	Any	Yes
HDF5	k-section	k-section	Any	Yes
HDF5	k-section	Hilbert	—	No

ond, each rank reads its assigned AMR files, determines the owner of each grid, and an `MPI_ALLTOALLV` exchange routes grids to their new owners. Lastly, hydro and gravity data are redistributed using the same exchange metadata. Particle files are filtered by position into the local k-section domain.

7.3 Format Compatibility

Table 2 summarises the supported input–output combinations for variable-$N_{\rm cpu}$ restart. The k-section decomposition can restore from any input format and rank count because the spatial tree is rebuilt from scratch. Conversely, restoring to Hilbert ordering requires the input to have been written with Hilbert ordering, and the binary format further restricts restarts to the original $N_{\rm cpu}$.

8 GPU-ACCELERATED SOLVERS

Certain compute-intensive routines, namely the Godunov solver, gravity force computation, hydrodynamic synchronisation, CFL timestep, prolongation, and radiative cooling, are amenable to GPU acceleration. Rather than offloading entire time steps to the GPU, cuRAMSES adopts a *hybrid dispatch* model in which OMP threads dynamically choose between CPU and GPU execution at runtime.

8.1 Dynamic Dispatch Model

At the start of each parallel region, each OMP thread attempts to acquire a GPU stream slot via an atomic counter. Threads that succeed accumulate grid data into a **batched grid buffer** of configurable size (by default, 4096 grids) and launch GPU kernels asynchronously when the buffer becomes full. Threads that do not acquire a slot execute the standard CPU code path. The `schedule(dynamic)` clause ensures load balancing, so that if a GPU thread is waiting for kernel completion, remaining loop iterations are picked up by CPU threads.

This design requires no code duplication, as the CPU path is the original Fortran subroutine, and the GPU path is an alternative branch within the same `OpenMP do` loop.

8.2 Batched Grid Buffering

GPU kernel launch latency ($\sim$10–50 μs) is reduced by batching, whereby each GPU thread accumulates the full stencil data for many grids before launching a single kernel covering all accumulated grids. For the Godunov solver, the GPU pipeline executes five kernels in sequence of primitive variable conversion, slope computation, Riemann tracing, flux computation, and artificial diffusion.

A key optimisation is the **on-device flux reduction** that computes the conservative update entirely on the GPU. Instead of transferring the full flux array back to the host ($\sim$98 MB per flush), the kernel reduces fluxes into compact per-grid output arrays, reducing the device-to-host transfer to $\sim$5 MB per flush, a 20-times reduction in PCIe bandwidth.

8.3 GPU-Resident Mesh Data

We note that the GPU benchmarks presented in this section are necessarily limited in scale. Multi-GPU clusters large enough to test production-size cosmological simulations remain scarce and prohibitively expensive for most research groups. The results below should therefore be interpreted as a proof of concept rather than a definitive GPU scaling assessment.

The initial GPU implementation gathers the stencil data for each batch of grids on the CPU, copies it to the device, and copies the results back after the kernel finishes. Profiling reveals that this host-to-device (H2D) transfer accounts for about majority of the total GPU time, completely negating the kernel speedup.

We therefore adopt a *GPU-resident mesh* strategy in which the full AMR mesh arrays (`uold`, `unew`, `phi`, `f`, `son`) are uploaded to the device once at the beginning of each level update. The gather phase, which formerly assembled the stencil on the host, is moved to a GPU kernel that reads directly

Table 3. GPU performance comparison on H100 NVL (2 ranks × 4 threads, 2 coarse steps). "MG base" is the base-level multigrid solve and "MG AMR" is the fine-level AMR correction. "Full D2H/H2D" transfers the entire `phi` array every MG iteration; "Halo-only" transfers only boundary cells during smoothing but still requires a full transfer for interpolation.

Mode	Total (s)	MG base (s)	MG AMR (s)	Godunov (s)
CPU-only	45.9	10.2	(incl.)	9.5
GPU full D2H/H2D	66.6	23.1	11.5	7.4
GPU halo-only	82.0	30.5	17.6	8.4

from device-resident arrays. After the Godunov sweep the scatter kernel writes updated fluxes back to `unew` on the device, and only the modified portion is copied back to the host.

This reorganisation reduces the H2D fraction almost negligible (less than 10 % of wall-clock time). The CPU-side gather time drops from 70.7 s to 2.1 s (34×) and the H2D transfer from 6.6 s to 0.24 s (27×). The approach requires one `MPI` rank per GPU because the mesh upload consumes 4–9 GB of device memory per rank. When the device memory is insufficient the code falls back automatically to CPU execution.

With the Godunov solver efficiently offloaded, we next turn to the other major compute kernel, the multigrid Poisson solver, which dominates the remaining runtime.

8.4 GPU Multigrid Solver

We extend the GPU acceleration to the multigrid (MG) Poisson solver. The red-black Gauss–Seidel smoother and residual computation are ported to CUDA kernels, with the device-side data (`phi`, `f`, neighbour arrays) totalling approximately 6.4 GB.

A *halo-only* exchange — transferring only the ghost-zone boundary cells between host and device instead of the full `phi` array — reduces per-sweep PCIe volume from ∼2.4 GB to ∼6 MB. However, the interpolation step between AMR levels still requires a full `phi` device-to-host transfer at each V-cycle.

8.5 GPU Performance Assessment

Table 3 summarises the GPU performance on the *Syntax* cluster[1], which hosts NVIDIA H100 NVL GPUs (93.1 GB HBM3 per device, PCIe Gen5), using 2 `MPI` ranks × 4 OMP threads, running 2 coarse steps of the Cosmo512 test (Section 9.1).

The Godunov solver benefits modestly from GPU offloading (9.5 s → 7.4–8.4 s), but the multigrid solver is consistently *slower* on the GPU, with the base-level solve degrading from 10.2 s to 30.5 s (about 3 times slower). The root cause is the PCIe bandwidth bottleneck. Even on the H100 NVL with PCIe Gen5, pageable memory throughput is 9.2 GB/s and pinned memory reaches only 13.6–22.4 GB/s. The MG solver's coarse-level solve and the interpolation/restriction operators remain on the CPU, forcing a full `phi` (a potential array) device-to-host and host-to-device transfer at every V-cycle iteration. This synchronisation cost overwhelms the computational savings from the GPU kernels.

The fundamental difficulty is that AMR multigrid requires level-by-level processing with inter-level transfers that break GPU data residency. A net GPU speedup would require full device-residency of the entire V-cycle. For the current code structure, CPU-only optimizations (Section 5) remain more effective.

9 PERFORMANCE RESULTS

9.1 Test Configuration

We use three test configurations of increasing size, referred to throughout as **Cosmo256**, **Cosmo512**, and **Cosmo1024**.

The **Cosmo256** test is designed for code consistency verification. It has 200 M dark matter particles in a periodic box of side $256\,h^{-1}$ Mpc, initialised at $z = 29.5$ with MUSIC (Hahn & Abel 2011). The base AMR grid is 256^3 (`levelmin`=8) with adaptive refinement up to `levelmax`=10. The simulation is restarted from an HDF5 output at coarse step 5 and evolved to step 10 (5 coarse steps). This small problem fits comfortably in a single node and enables rapid regression testing of every optimization against a reference Hilbert-ordering run.

The **Cosmo512** test, used for intra-node strong scaling (Section 9.3), is a cosmological zoom-in simulation sized to fit within a single compute node. It contains 135.7 M dark matter particles and 3.2×10^4 sink particles in the same $256\,h^{-1}$ Mpc box with ΛCDM cosmology ($\Omega_b = 0.04$). The base grid is 512^3 (`levelmin`=9) with `levelmax`=17; at the restart epoch (coarse step 241) levels 9–14 are active. The physics suite includes radiative cooling (Haardt & Madau background; Haardt & Madau 2012), star formation, Type II supernova (SNII) and AGN feedback, Bondi black hole accretion, and BH spin with magnetically arrested disc (MAD) jets. Initial conditions are generated with MUSIC at $z = 29.5$. Each `MPI` rank allocates $N_{\rm gridmax} = 5$ M grids (88 M total grids across all ranks), and the run is restarted from an HDF5 output at coarse step 241.

The **Cosmo1024** test, used for multi-node scaling (Section 9.6), is a larger cosmological zoom-in simulation designed to evaluate parallel efficiency across multiple compute nodes. It contains 1024^3 (≈ 1.07 B) dark matter particles in a $256\,h^{-1}$ Mpc periodic box with the same ΛCDM cosmology. The base grid is 1024^3 (`levelmin`=10) with `levelmax`=18; at the restart epoch (coarse step 218, $z \approx 5.2$) levels 10–15 are active, comprising ∼ 537 M AMR grids and ∼ 4.2 M stellar particles. The full physics suite is identical to Cosmo512. The run is restarted from an HDF5 output at coarse step 218 using the variable-$N_{\rm cpu}$ restart feature.

The Cosmo256 and Cosmo512 tests are run on a dual-socket AMD EPYC 7543 node (64 physical cores, 128 threads) with 1 TB of DDR4 memory. The Cosmo1024 test is run on the *Grammar* cluster[2] (8 nodes, dual-socket AMD EPYC 7543 per node, 64 cores/node, 512 GB RAM per node, 100 Gbps InfiniBand interconnect). This configuration is designed to evaluate parallel efficiency at a scale representative of production cosmological simulations, where billion-particle zoom-in runs routinely span multiple nodes.

[1] https://cac.kias.re.kr/systems/clusters/syntax

[2] https://cac.kias.re.kr/systems/clusters/grammar

9.2 Conservation Verification

Every optimization is verified against a Hilbert-ordering reference run. The comparison encompasses the full set of global diagnostics: energy conservation error $e_{\rm cons}$, gravitational potential energy $e_{\rm pot}$, kinetic energy $e_{\rm kin}$, internal energy $e_{\rm int}$, total mass $m_{\rm tot}$, number of stars formed $n_\star$, number of sink particles $n_{\rm sink}$, and the total number of created AMR grids. The k-section decomposition, Morton hash table, and their combination produce diagnostics identical to the reference ($e_{\rm cons} = 3.77 \times 10^{-3}$, $e_{\rm pot} = -1.88 \times 10^{-6}$, $e_{\rm kin} = 1.23 \times 10^{-6}$; all other quantities match to machine precision). The MG-optimized configuration yields $e_{\rm cons} = 3.79 \times 10^{-3}$ (0.5 per cent higher), attributable to relaxed boundary synchronisation in the merged red-black sweep, well within the solver's convergence tolerance.

Achieving bit-identical star and sink counts across different domain decompositions requires a thread-safe random number generator. The standard Fortran `RANDOM_NUMBER` intrinsic uses per-image state that depends on the thread scheduling order, introducing non-determinism under OpenMP. We replace it with a deterministic generator seeded by the global grid cell index, so that each cell receives the same random draw regardless of the number of `MPI` ranks or OpenMP threads. This eliminates the sole source of run-to-run variation in star formation stochasticity.

All optimizations operate purely at the data-structure and communication level, leaving the discrete operators unchanged.

9.3 Strong Scaling

We separate the scaling analysis into two complementary parts. The full-physics application benchmark in this section tests intra-node strong scaling with all solver components active, providing precise per-component attribution. The multi-node communication scaling behaviour, which is governed by the k-section domain decomposition and sparse P2P exchange patterns, is characterised independently through dedicated microbenchmarks on a 16-node cluster in Section 9.7.1.

We measure strong scaling by restarting the Cosmo512 simulation from an HDF5 output. The test problem comprises 54 M AMR grids (levels 9–14) and 135.7 M particles including 3.2×10^4 sink particles, with full physics enabled (radiative cooling, star formation, SNII/AGN feedback). The output was written at coarse step 241 with 12 `MPI` ranks, and we run 2 additional coarse steps to step 243. The variable-$N_{\rm cpu}$ restart feature (§7) allows the output to be read with any number of `MPI` ranks. A forced `load_balance` on the first coarse step ensures optimal grid distribution before timing begins. The test platform has two AMD EPYC 7543 processors (64 cores, 128 threads) and 1 TB of shared memory; all runs use `OMP_NUM_THREADS=1`.

The wall-clock time and per-component timer averages for $N_{\rm cpu} = 1$–64 are measured. All configurations yield energy conservation errors $e_{\rm cons}$ of $\mathcal{O}(10^{-4})$, with small variations due to floating-point summation order changes across different domain decompositions.

Several scaling trends are evident:

- *Particle operations* scale nearly ideally from 1 to 64 ranks: 538 s → 14 s, a 38.4× speedup for 64× the ranks.
- *Multigrid Poisson solver* dominates the total time (∼37%), scaling from 4034 s (1 rank) to 106 s (64 ranks). The 38.2× speedup is near-ideal, reflecting the effectiveness of the Morton hash-based neighbour lookup and precomputed cache arrays.
- *Godunov solver* scales 58.4× from 1 to 64 ranks (2536 s → 43 s). The super-linear speedup arises because multi-rank runs benefit from cache effects, since each rank processes a smaller working set that fits in the L3 cache.
- *Sink particle operations* scale well up to 24 ranks (1346 s → 69 s, 19.5×) but slow beyond 32 ranks due to the global ALLREDUCE operations in AGN feedback.
- *Load balancing* overhead converges to ∼16 s beyond 32 ranks. This cost is spread over `nremap` coarse steps (every 5 steps in production).
- The *overall speedup* reaches 33.9× at 64 ranks relative to the single-rank baseline, demonstrating excellent scaling efficiency (53%) on this shared-memory node.

Figure 7 visualises these trends. Panel (a) shows the elapsed time and per-component timer values as a function of $N_{\rm cpu}$ on a log–log scale. All major components follow the ideal scaling line closely up to ∼16 ranks, with gradual deviation at higher rank counts due to communication overhead. Panel (b) shows the speedup relative to the single-rank baseline: particle operations and the flag routine achieve near-ideal scaling, while the elapsed time reaches 33.9× at 64 ranks.

9.4 OpenMP Thread Scaling

To evaluate intra-rank parallelism we fix $N_{\rm cpu} = 4$ and vary `OMP_NUM_THREADS` from 1 to 30, using the same Cosmo512 problem as Section 9.3. The total core count ranges from 4 to 120, and the physical core limit of the dual-socket node is 64.

All runs conserve energy to $e_{\rm cons} \leq 6.56 \times 10^{-4}$.

Several trends distinguish the OpenMP scaling from the `MPI`only results:

- *Multigrid Poisson solver* benefits the most from threading, improving from 1085 → 121 s at 16 threads (8.9×), with continued gains beyond 16 threads reaching 10.5× at 30 threads. The precomputed neighbour arrays and fused residual loops (Section 5) expose substantial loop-level parallelism.
- *Godunov solver* scales 11.4× from 1 to 16 threads (610 → 54 s). Beyond 16 threads, performance degrades slightly due to memory bandwidth saturation and NUMA effects.
- *Sinks and Poisson* exhibit modest scaling (3.0× and 3.0× at 16 threads) because sink operations involve global reductions and serial sections that limit parallelism.
- *Overall speedup* plateaus at 5.8× with 16 threads (64 cores), limited by the serial fraction of `MPI` communication and sink-particle operations. Beyond the physical core count, oversubscription yields no further improvement.

Figure 8 compares per-component timers and speedup curves as a function of $N_{\rm omp}$. The vertical dotted line marks the physical core limit (64 cores). Panel (a) shows that the MG solver and Godunov components track the ideal scaling line most closely, while sinks and Poisson saturate early. Panel (b) confirms that the overall speedup reaches a ceiling

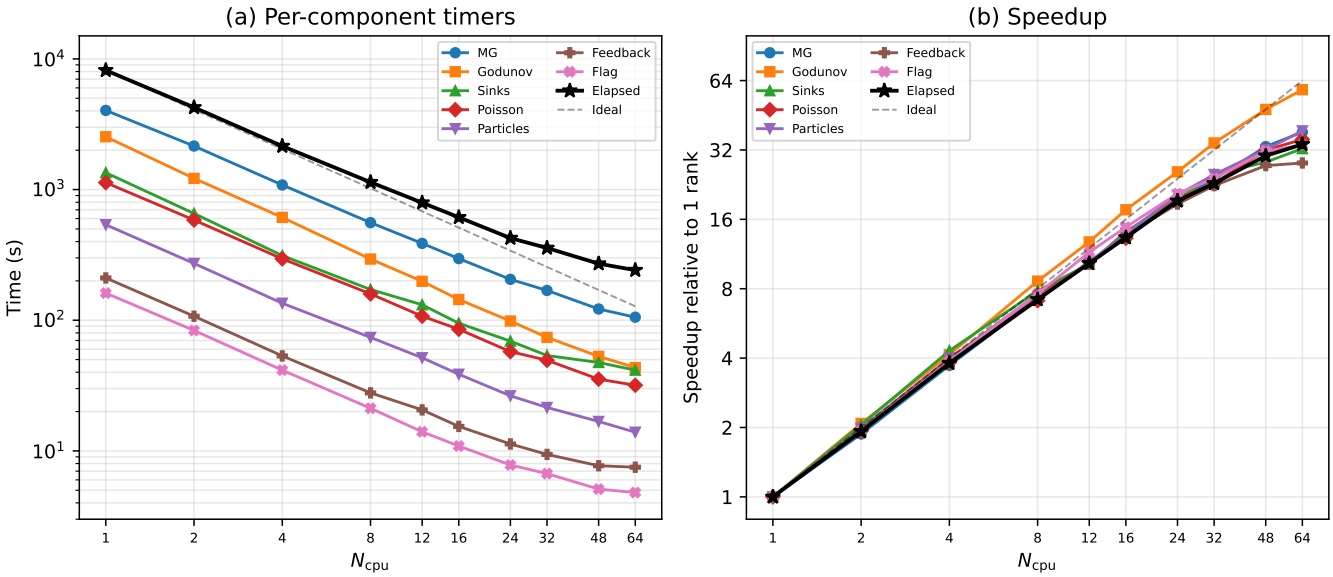

Figure 7. Strong scaling of CURAMSES on a dual-socket AMD EPYC 7543 node (64 cores) with a 135.7 M particle cosmological simulation including full physics. (a) Elapsed time and per-component wall-clock times versus $N_{\rm cpu}$. The dashed line shows ideal scaling from the single-rank baseline. (b) Speedup relative to 1 rank. Particle operations and the flag routine scale near-ideally, while the overall speedup reaches 33.9× at 64 ranks (53% parallel efficiency).

Table 4. Memory load balance for the strong-scaling test (54 M grids, 135.7 M particles). $M_{\rm min}$ and $M_{\rm max}$ are the minimum and maximum per-rank resident memory after the final load-balance remap (step 243). The ratio $M_{\rm max}/M_{\rm min}$ measures memory imbalance. The level-10 grid counts illustrate the per-rank work imbalance at the most populated refined level (19.5 M grids total).

$N_{\rm cpu}$	$M_{\rm min}$ (Gb)	$M_{\rm max}$ (Gb)	$\frac{M_{\rm max}}{M_{\rm min}}$	Lv-10 grids/rank ($\times 10^3$) min	max	ratio
1	147.2	147.2	1.000		—	
2	87.0	87.4	1.005	9120	10338	1.13
4	50.1	50.5	1.008	4546	5210	1.15
8	28.0	28.6	1.021	2200	2675	1.22
12	20.9	21.4	1.024	1370	1828	1.33
16	16.7	17.1	1.024	1021	1376	1.35
24	12.5	13.1	1.048	671	937	1.40
32	10.7	11.1	1.037	521	711	1.36
48	8.2	8.6	1.049	306	479	1.56
64	8.2	8.6	1.049	225	362	1.61

near 6×, indicating that further performance gains require additional `MPI` ranks rather than more threads per rank.

9.5 Memory-Weighted Load Balancing

The memory-weighted cost function (equation 4) assigns $w_{\rm grid}$ automatically from `nvar` (2256 B for `nvar`=14 in our test) and $w_{\rm part} = 12$ bytes per particle. Table 4 quantifies the load balance achieved by this scheme across the strong-scaling runs described in the preceding section.

The memory-weighted balancer keeps the per-rank memory imbalance remarkably low, with $M_{\rm max}/M_{\rm min} \leq 1.05$ for all rank counts tested, from 2 to 64. This is achieved despite substantial grid-count imbalance at the most populated refined level (level 10, containing 19.5 M grids), where the per-rank max/min ratio grows from 1.13 at 2 ranks to 1.61 at 64 ranks because the k-section sub-domains become smaller relative to the AMR clustering scale.

The low memory imbalance is key for production runs because each rank must pre-allocate arrays sized to its local $N_{\rm gridmax}$. A high imbalance forces over-allocation on lightly loaded ranks. With memory-weighted balancing, the 64-rank run requires only 8.6 GB per rank at peak, compared with the 147.2 GB needed in a single-rank run, a factor of 17.1 reduction, close to the ideal 64× scaling modulo the ∼4 GB fixed per-rank overhead (`MPI` buffers, hash tables, coarse grid). The fixed overhead explains why per-rank memory saturates at ∼8 GB for 48 and 64 ranks.

The load-balance remap itself costs 16–26 s for $N_{\rm cpu} \geq 8$, growing from 2.3 to 5.1 per cent of the total runtime as the computation shrinks under strong scaling, a modest price for near-perfect memory balance. Figure 9 visualizes these trends.

9.6 Multi-Node Scaling (Cosmo1024)

The preceding single-node tests demonstrate intra-node scaling efficiency but cannot probe the inter-node communication overhead that dominates production runs. We therefore perform a multi-node strong-scaling study using the Cosmo1024 configuration on the *Grammar* cluster (8 nodes, dual-socket AMD EPYC 7543, 64 cores/node, 512 GB RAM/node, 100 Gbps InfiniBand).

9.6.1 MPI rank scaling

We restart the Cosmo1024 simulation from its HDF5 output at coarse step 218 ($z \approx 5.2$) and run 5 additional coarse steps,

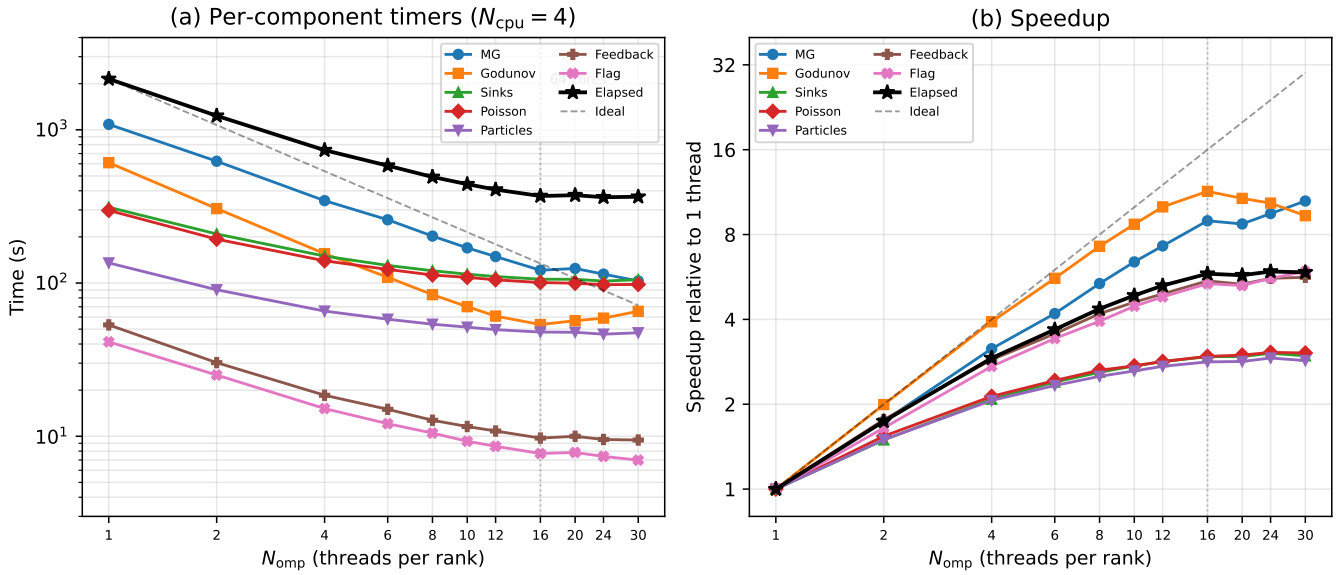

Figure 8. OpenMP thread scaling of CURAMSES with $N_{cpu} = 4$ MPI ranks on a dual-socket AMD EPYC 7543 node (64 cores). (a) Per-component wall-clock times versus N_{omp}. The dashed line shows ideal scaling from the single-thread baseline. The vertical dotted line marks the physical core limit (16 threads × 4 ranks = 64 cores). (b) Speedup relative to 1 thread. The MG solver achieves 10.5× speedup, while the overall elapsed time plateaus at ∼5.8×.

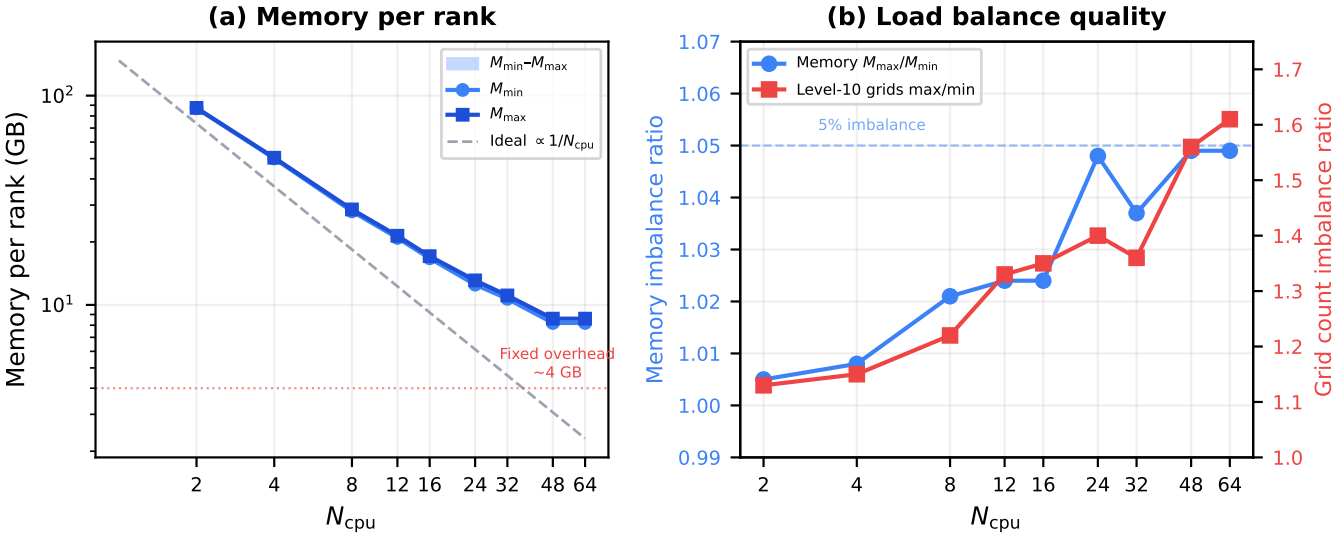

Figure 9. Memory-weighted load balance across the strong-scaling test suite. (a) Per-rank memory: M_{min} and M_{max} follow the ideal $1/N_{cpu}$ scaling (dashed grey) until a fixed per-rank overhead of ∼4 GB dominates at high rank counts. (b) Load balance quality: the memory imbalance ratio M_{max}/M_{min} (blue, left axis) remains below 5 per cent for all rank counts tested (2–64), while the level-10 grid count imbalance (red, right axis) grows to 1.61 at 64 ranks, demonstrating that the memory-weighted cost function successfully decouples memory balance from grid-count balance.

discarding the first step (which includes the `load_balance` remap) as warm-up. The number of MPI ranks is varied as $N_{cpu} = 8, 16, 32, 64$, with `OMP_NUM_THREADS`=8 and $N_{cpu}/8$ ranks per node, so that the total core count doubles from 64 (1 node equivalent) to 512 (8 nodes). The variable-N_{cpu} restart feature allows each rank count to read the same checkpoint.

Figure 10 shows the elapsed time and speedup as a function of total cores. Despite the inter-node communication overhead, the multi-node run achieves good parallel efficiency. The hierarchical k-section exchange (Section 3) keeps the number of communication partners bounded at $O(\log P)$, preventing the all-to-all scaling bottleneck that would otherwise appear at high rank counts with Hilbert ordering.

fig_cosmo1024_mpi_scaling.pdf

Figure 10. Multi-node strong scaling of cuRAMSES (Cosmo1024) on the Grammar cluster (8 nodes, 64 cores/node). (a) Wall-clock time per coarse step versus total cores (64–512). The dashed line shows ideal scaling from the 64-core baseline. (b) Speedup relative to 64 cores. The upper axis shows the equivalent number of nodes.

fig_cosmo1024_omp_scaling.pdf

Figure 11. Hybrid MPI/OpenMP scaling of cuRAMSES (Cosmo1024) on 8 nodes (512 cores total), varying the MPI/OMP split ($N_{\rm cpu} \times N_{\rm omp} = 512$). (a) Per-step wall-clock time decomposed into solver components. The vertical red line marks the production configuration ($N_{\rm cpu} = 64$, $N_{\rm omp} = 8$). (b) Speedup relative to the pure-MPI baseline ($N_{\rm omp} = 1$). The vertical dotted line marks the physical core limit (8 threads/rank).

9.6.2 MPI/OpenMP hybrid scaling

To identify the optimal MPI/OpenMP balance, we fix the total resource at 8 nodes (512 cores) and vary the MPI/OMP split: $N_{\rm cpu} \times N_{\rm omp} = 512$, with $N_{\rm omp}$ ranging from 1 (pure MPI, 512 ranks) to 64 (8 ranks, 64 threads each). Each configuration allocates $N_{\rm cpu}/8$ ranks per node, so the per-node memory footprint varies inversely with $N_{\rm cpu}$ while the total compute budget remains constant.

Figure 11 shows the per-step wall-clock time and relative speedup as a function of $N_{\rm omp}$. The vertical dotted line marks the physical core limit per rank (8 threads $\times$ 8 ranks/node = 64 cores/node). The production configuration ($N_{\rm cpu} = 64$, $N_{\rm omp} = 8$) is highlighted. Configurations with fewer MPI ranks and more threads reduce communication overhead but sacrifice intra-rank parallelism due to serial sections in sink operations and load balancing.

9.7 Communication Microbenchmarks

To validate the communication advantages of the k-section framework at scale, we design and execute a suite of dedicated microbenchmarks on the *Grammar* cluster (16 nodes, dual-socket AMD EPYC 7543, 64 cores/node, 512 GB RAM/node, 100 Gbps InfiniBand, Intel MPI 2021.17). All benchmarks are self-contained single-source Fortran programs using plane-by-plane ownership computation [$O(N^2)$ memory per rank] so that grid sizes up to $N = 2048$ are feasible.

9.7.1 Ghost zone exchange

This benchmark constructs a uniform N^3 periodic grid, assigns cell ownership via either Hilbert or k-section domain decomposition, and measures ghost zone exchange time for three communication backends: *P2P* (non-blocking MPI_ISEND/IRECV), MPI_ALLTOALLV, and the hierarchical k-section exchange described in Section 3. Each cell carries $n_{\rm var} = 5$ double-precision variables (matching RAMSES hydro), and timings are averaged over 3 repeats.

Results for $N = 1024$ (10^9 cells) on 64–1024 MPI ranks show the following trends:

- The ghost list *build time* is dominated by the Hilbert curve computation ($\sim$453–534 s, rising at high rank counts) versus the k-section tree traversal which scales from 200 s (64 ranks) to 354 s (1024 ranks). The k-section build is 1.5–2.3$\times$ faster across all rank counts.
- The *exchange time* decreases with rank count as the per-rank ghost volume shrinks. P2P and ALLTOALLV are effectively tied (7–42 ms at 64–1024 ranks), while the hierarchical k-section exchange is $\sim$5–10$\times$ slower (52–368 ms) due to its $O(\log P)$ serialised stages. The gap narrows at high rank counts: at 1024 ranks the k-section exchange drops to 52 ms versus 7 ms for P2P. The k-section exchange is designed for variable-size AMR communications where all-to-all collectives become prohibitive. On this uniform-grid test its overhead is expected.
- The average number of communication partners is constant at 6.0 for k-section across all rank counts, compared to 6.0–9.1 for Hilbert. Non-power-of-two rank counts (96, 192,

320, 448) increase the Hilbert neighbour count to 8.4–9.1, while k-section remains at exactly 6, reflecting the compact cuboid sub-domains produced by recursive bisection.

• The surface-to-volume ratio is consistently lower for k-section than for Hilbert at non-power-of-two ranks (e.g. 0.027 vs. 0.033 at 96 ranks), confirming that the cuboid decomposition minimises ghost volume.

9.7.2 MPI collective performance

A second benchmark profiles the raw performance of five `MPI` collectives, namely `MPI_ALLREDUCE`, a manual binary-tree allreduce (reduce to root then broadcast, $O(\log_2 P)$ stages), `MPI_BCAST`, neighbour-to-neighbour `MPI_SENDRECV` (1D ring), and `MPI_ALLTOALL`, over message sizes from 1 to 10^7 doubles (8 B to 80 MB). Each test executes a warmup round followed by 100 timed repeats; the per-repeat maximum across all ranks is recorded. These measurements provide a baseline for interpreting the dominant communication costs in cuRAMSES.

9.7.3 Load balance quality

To isolate the impact of domain decomposition geometry on load balance, a third benchmark assigns non-uniform cell weights drawn from 20 Gaussian clusters (amplitude 10–100, width $\sigma = 0.02$–0.10) superposed on a unit background. For each DD method (Hilbert vs. k-section) the benchmark reports (i) the weight imbalance ratio $W_{\max}/W_{\mathrm{mean}}$, (ii) the ghost cell ratio (fraction of owned cells with at least one face neighbour on a different rank), and (iii) the number of `MPI` communication partners per rank.

9.7.4 Poisson solver

The fourth benchmark compares three iterative solvers (Jacobi, red-black Gauss-Seidel, and unpreconditioned conjugate gradient) on a distributed N^3 periodic grid with slab decomposition in z. The source term is $f = -12\pi^2 \sin(2\pi x)\sin(2\pi y)\sin(2\pi z)$, giving an exact solution $\phi = \sin(2\pi x)\sin(2\pi y)\sin(2\pi z)$. Each solver separates computation (stencil update) from communication (ghost plane exchange via `MPI_SENDRECV`), enabling direct measurement of the compute/communication ratio. CG achieves machine-precision convergence in $O(N)$ iterations versus the $O(N^2)$ scaling of Jacobi and Gauss-Seidel, quantifying the benefit of the Krylov approach used in cuRAMSES' multigrid V-cycle smoother.

10 CONCLUSIONS

We have presented cuRAMSES, a set of algorithmic and implementation improvements to the RAMSES cosmological AMR code that collectively address the key scaling bottlenecks, namely communication overhead, memory consumption, solver efficiency, and hardware utilisation, encountered in large-scale cosmological simulations. While previous efforts such as OMP-RAMSES (Lee et al. 2021) and RAMSES-yOMP (Han et al. 2026) introduced `MPI`+OpenMP hybrid parallelism, cuRAMSES extends this to a three-level `MPI`+OpenMP+CUDA paradigm suited to the GPU-dominated architectures of current and upcoming exascale supercomputers. The main contributions are:

(i) **Recursive k-section domain decomposition** (Section 2): hierarchical `MPI` exchange with $\mathcal{O}(\sum_l k_l)$ communications, substituting all `MPI_ALLTOALL` calls, with memory-weighted load balancing reducing peak-to-mean imbalance to <5%.

(ii) **Morton key hash table** (Section 4): $\mathcal{O}(1)$ neighbour lookup replacing the `nbor` array, saving >190 MB per rank.

(iii) **Memory optimizations**: on-demand allocation saves >1 GB per rank (Appendix C).

(iv) **Multigrid solver optimizations** (Section 5): merged red-black sweeps and fused residual-norm reduce MG communication by 44 per cent, lowering the Poisson share from 50 to 40 per cent.

(v) **FFTW3 direct Poisson solver** (Section 5.2): 8.3× speedup at the base level with sparse P2P exchange.

(vi) **Feedback spatial binning** (Section 6): ∼260× SNII and 30× AGN speedups.

(vii) **Variable-N_{cpu} restart** (Section 7): HDF5 and binary I/O with arbitrary rank counts.

(viii) **GPU acceleration** (Section 8): GPU-resident mesh reduces H2D transfer to 5.7 per cent, but the MG solver is 3× slower on GPU due to per-V-cycle PCIe transfers; CPU-only optimization remains more effective for AMR multigrid.

All optimizations preserve conservation to machine precision (Section 9.2). Every global diagnostic — e_{cons}, e_{pot}, e_{kin}, e_{int}, m_{tot}, $n_\star$, n_{sink}, and the total number of AMR grids — is verified to match the Hilbert-ordering reference run, with only a 0.5 per cent shift in e_{cons} for the merged red-black sweep. The use of a deterministic, cell-index-seeded random number generator ensures that star formation stochasticity is independent of the domain decomposition and thread scheduling.

The techniques presented here — Morton hash table, k-section decomposition, spatial binning, and auto-tuning `MPI` backends — are general and applicable to other octree AMR codes beyond RAMSES. The FFTW3 base-level solver opens the door to modified-gravity cosmologies (e.g. massive neutrinos, coupled dark energy) via Fourier-space transfer functions, without altering the real-space AMR infrastructure.

The GPU acceleration experiment (Section 8) demonstrates that a hybrid CPU+GPU approach is viable for the Godunov solver but counterproductive for AMR multigrid, where per-V-cycle PCIe transfers negate the kernel speedup. Achieving efficient GPU utilisation for AMR multigrid remains an open challenge requiring full device-residency of the V-cycle, ideally with GPU-native restriction, prolongation, and coarse-level solve.

Strong scaling on a single 64-core node reaches 33.9× speedup (53 per cent parallel efficiency), while OpenMP thread scaling achieves 5.8× with 16 threads per rank. The variable-N_{cpu} restart capability, supporting both HDF5 and binary formats, provides operational flexibility for production campaigns where hardware availability changes between runs.

cuRAMSES is in production use for the Horizon Run 5

cosmological simulation project and will be made publicly available upon completion of the benchmark campaign.

ACKNOWLEDGEMENTS

This work was supported by the Korea Institute for Advanced Study. Computational resources were provided by the KIAS Center for Advanced Computation. The author thanks Romain Teyssier for the public release of the RAMSES code and the RAMSES developer community for continued maintenance and improvements.

DATA AVAILABILITY

The modified code is available at `https://github.com/kjhan0606/cuRAMSES`. Test configurations and analysis scripts will be shared upon reasonable request to the author.

REFERENCES

Adams M. F., 2001, in Proc. SC01. ACM, Denver, doi:10.1145/582034.582038
Brandt A., 1977, Mathematics of Computation, 31, 333, doi:10.1090/S0025-5718-1977-0431719-X
Briggs W. L., Henson V. E., McCormick S. F., 2000, A Multigrid Tutorial, 2nd edn. SIAM, Philadelphia
Bryan G. L., et al., 2014, ApJS, 211, 19, doi:10.1088/0067-0049/211/2/19, arXiv:1307.2265
Davé R., Anglés-Alcázar D., Narayanan D., Li Q., Rafieferantsoa M. H., Appleby S., 2019, MNRAS, 486, 2827, doi:10.1093/mnras/stz937, arXiv:1901.10203
Dubois Y., Devriendt J., Slyz A., Teyssier R., 2012, MNRAS, 420, 2662, doi:10.1111/j.1365-2966.2011.20142.x, arXiv:1108.0110
Dubois Y., et al., 2014, MNRAS, 444, 1453, doi:10.1093/mnras/stu1227, arXiv:1402.1165
Dubois Y., et al., 2021, A&A, 651, A109, doi:10.1051/0004-6361/202039429, arXiv:2009.10578
Frigo M., Johnson S. G., 2005, Proc. IEEE, 93, 216, doi:10.1109/JPROC.2004.840301
Frontiere N., et al., 2025, in Proc. SC '25, ACM, New York, doi:10.1145/3712285.3771786, arXiv:2510.03557
Guillet T., Teyssier R., 2011, J. Comput. Phys., 230, 4756, doi:10.1016/j.jcp.2011.02.044, arXiv:1104.1703
Haardt F., Madau P., 2012, ApJ, 746, 125, doi:10.1088/0004-637X/746/2/125, arXiv:1105.2039
Habib S., et al., 2016, New Astron., 42, 49, doi:10.1016/j.newast.2015.06.003, arXiv:1410.2805
Hahn O., Abel T., 2011, MNRAS, 415, 2101, doi:10.1111/j.1365-2966.2011.18820.x, arXiv:1103.6031
Han S., et al., 2026, A&A, 705, A169, doi:10.1051/0004-6361/202556291, arXiv:2507.06301
Hopkins P. F., Kereš D., Oñorbe J., Faucher-Giguère C.-A., Quataert E., Murray N., Bullock J. S., 2014, MNRAS, 445, 581, doi:10.1093/mnras/stu1738, arXiv:1311.2073
Hopkins P. F., 2015, MNRAS, 450, 53, doi:10.1093/mnras/stv195, arXiv:1409.7395
Hopkins P. F., et al., 2018, MNRAS, 480, 800, doi:10.1093/mnras/sty1690, arXiv:1702.06148
Ishiyama T., et al., 2021, MNRAS, 506, 4210, doi:10.1093/mnras/stab1755, arXiv:2007.14720
Kim J., Park C., Rossi G., Lee S. M., Gott III J. R., 2011, J. Korean Astron. Soc., 44, 217, doi:10.5303/JKAS.2011.44.6.217
Kim J., Park C., L'Huillier B., Hong S. E., 2015, J. Korean Astron. Soc., 48, 213, doi:10.5303/JKAS.2015.48.4.213, arXiv:1508.05107
Klypin A., Prada F., 2019, MNRAS, 489, 1684, doi:10.1093/mnras/stz2217
Klypin A., Prada F., 2018, MNRAS, 478, 4602, doi:10.1093/mnras/sty1340
Knuth D. E., 1997, The Art of Computer Programming, Vol. 3: Sorting and Searching, 2nd edn. Addison-Wesley, Reading, MA
Lee J., et al., 2021, ApJ, 908, 11, doi:10.3847/1538-4357/abd08b, arXiv:2006.01039
Libeskind N. I., et al., 2018, MNRAS, 473, 1195, doi:10.1093/mnras/stx1976, arXiv:1705.03021
Maksimova N. A., Garrison L. H., Eisenstein D. J., Hadzhiyska B., Bose S., Satterthwaite T. P., 2021, MNRAS, 508, 4017, doi:10.1093/mnras/stab2484, arXiv:2110.11398
Morton G. M., 1966, A Computer Oriented Geodetic Data Base and a New Technique in File Sequencing. IBM, Ottawa
Nelson D., et al., 2019, Comput. Astrophys. Cosmol., 6, 2, doi:10.1186/s40668-019-0028-x, arXiv:1812.05609
Pillepich A., et al., 2018, MNRAS, 473, 4077, doi:10.1093/mnras/stx2656, arXiv:1703.02970
RAMSES Team, 2024, RAMSES User's Guide, `https://ramses-organisation.readthedocs.io`
Samet H., 2006, Foundations of Multidimensional and Metric Data Structures. Morgan Kaufmann
Schaye J., et al., 2015, MNRAS, 446, 521, doi:10.1093/mnras/stu2058, arXiv:1407.7040
Springel V., Hernquist L., 2003, MNRAS, 339, 289, doi:10.1046/j.1365-8711.2003.06206.x, arXiv:astro-ph/0206393
Springel V., 2005, MNRAS, 364, 1105, doi:10.1111/j.1365-2966.2005.09655.x, arXiv:astro-ph/0505010
Springel V., 2010, MNRAS, 401, 791, doi:10.1111/j.1365-2966.2009.15715.x, arXiv:0901.4107
Teyssier R., 2002, A&A, 385, 337, doi:10.1051/0004-6361:20011817, arXiv:astro-ph/0111367
Trottenberg U., Oosterlee C. W., Schüller A., 2001, Multigrid. Academic Press
Tremmel M., Karcher M., Governato F., Volonteri M., Quinn T. R., Pontzen A., Anderson L., Bellovary J., 2017, MNRAS, 470, 1121, doi:10.1093/mnras/stx1160, arXiv:1607.02151
Wesseling P., 1992, An Introduction to Multigrid Methods. Wiley, Chichester
Vogelsberger M., et al., 2014, MNRAS, 444, 1518, doi:10.1093/mnras/stu1536, arXiv:1405.2921
Warren M. S., Salmon J. K., 1993, in Proc. Supercomputing '93. ACM, New York, p. 12, doi:10.1145/169627.169640
Weinberger R., et al., 2017, MNRAS, 465, 3291, doi:10.1093/mnras/stw2944, arXiv:1607.03486
Truelove J. K., Klein R. I., McKee C. F., Holliman J. H., Howell L. H., Greenough J. A., 1997, ApJ, 489, L179, doi:10.1086/310975

APPENDIX A: MORTON KEY ENCODING DETAILS

The Morton key interleaving for a single coordinate value v with B bits per coordinate ($B = 21$ for 64-bit keys, $B = 42$ for 128-bit keys) is computed by the following bit-manipulation loop:

Algorithm 1 Morton key encoding of (i_x, i_y, i_z)

Input: Integer coordinates (i_x, i_y, i_z)
Output: $3B$-bit Morton key M

$M \leftarrow 0$
for $b = 0$ **to** $B - 1$ **do**
  $M \leftarrow M \,|\, (\mathrm{bit}_b(i_x)\texttt{<<}3b)$
  $M \leftarrow M \,|\, (\mathrm{bit}_b(i_y)\texttt{<<}(3b+1))$
  $M \leftarrow M \,|\, (\mathrm{bit}_b(i_z)\texttt{<<}(3b+2))$
end for

The neighbour key computation decodes, shifts the appropriate coordinate, applies periodic wrapping, and re-encodes:

Algorithm 2 Morton neighbour key in direction j

Input: Morton key M, direction j, grid counts (n_x, n_y, n_z)
Output: Neighbour Morton key M' (or -1 if out of bounds)

$(i_x, i_y, i_z) \leftarrow \textsc{Decode}(M)$
Adjust i_d by ± 1 according to direction j
Apply periodic wrapping: $i_d \leftarrow i_d \bmod n_d$
$M' \leftarrow \textsc{Encode}(i_x, i_y, i_z)$

APPENDIX B: PER-LEVEL HASH TABLE AND REPLACEMENT FUNCTIONS

We maintain one open-addressing hash table (probing consecutive slots on collision) per AMR level, mapping Morton keys M to grid indices `igrid`.

The hash function first reduces the key to 64 bits (for 128-bit keys, an XOR fold of the upper and lower halves; for 64-bit keys, a direct cast), then applies multiplicative hashing with an additional mixing step:

$$t = \left[(h \times \phi_1)\hat{}(h\texttt{>>}16)\right] \times \phi_2,$$
$$h(M) = t\hat{}(t\texttt{>>}13), \tag{B1}$$

where h is the 64-bit reduced key, ^ is bitwise XOR, >> is a logical right bit-shift, $\phi_1 = 2654435761$ and $\phi_2 =$ `0x9E3779B97F4A7C15` are constants chosen for good bit mixing, and the table capacity is always a power of two to allow bitmask modular arithmetic. Collisions are resolved by linear probing, and the fill fraction is kept below 0.7 by automatic rehashing (doubling capacity).

The hash table is maintained incrementally.

- `morton_hash_insert` is called during `make_grid_coarse` and `make_grid_fine`.
- `morton_hash_delete` is called during `kill_grid`.
- A full rebuild is performed after defragmentation (`morton_hash_rebuild`).

A companion array `grid_level(igrid)` stores the AMR level of each grid, enabling Morton key computation from the grid index alone.

Two wrapper functions provide drop-in replacements for the original `nbor`-based access patterns.

- `morton_nbor_grid(igrid, ilevel, j)` returns the grid index of the same-level neighbour in direction j, replacing the pattern `son(nbor(igrid, j))`. The implementation computes the Morton key, shifts by direction, and looks up the result in the hash table.
- `morton_nbor_cell(igrid, ilevel, j)` returns the father cell index of the neighbour, replacing the pattern `nbor(igrid, j)`. For level 1 it returns the coarse cell index directly. For finer levels it computes the parent grid via the hash table at level $l - 1$ and the octant index from the coordinate parity.

The `nbor` array is reduced to `allocate(nbor(1:1, 1:1))` , effectively eliminated while maintaining compilation compatibility with any remaining references.

APPENDIX C: MEMORY SAVINGS DETAILS

C1 Hilbert Key and Histogram Array Elimination

When using k-section ordering, the Hilbert key array `hilbert_key(1:ncell)` is no longer needed for domain decomposition. We replace it with `allocate(hilbert_key(1:1))`, saving $16\,N_{\mathrm{gridmax}}\,2^{N_{\mathrm{dim}}}$ bytes under `QUADHILBERT` (128-bit keys stored as two 64-bit integers). For $N_{\mathrm{gridmax}} = 5\,\mathrm{M}$, this is approximately 640 MB.

The defragmentation routine, which previously required Hilbert keys for reordering, uses a local scratch array (`defrag_dp`) allocated only during the defragmentation pass and immediately deallocated.

Similarly, the arrays `bisec_ind_cell` and `cell_level`, each of size $N_{\mathrm{gridmax}}\,2^{N_{\mathrm{dim}}}$ integers (8 bytes), are used exclusively during load balancing to build the bisection histogram. We allocate them on entry to `init_bisection_histogram` and deallocate them after `cmp_new_cpu_map` returns, saving $16\,N_{\mathrm{gridmax}}\,2^{N_{\mathrm{dim}}} \approx 320\,\mathrm{MB}$ for $N_{\mathrm{gridmax}} = 5\,\mathrm{M}$. Since load balancing occurs only every `nremap` coarse steps, these arrays are absent during the vast majority of the simulation.

C2 Memory Savings Summary

The hash table memory cost is given by equation (**??**). The `grid_level` array adds $4\,N_{\mathrm{gridmax}}$ bytes. Compared to the original `nbor` cost of $48\,N_{\mathrm{gridmax}}$ bytes, the net savings are approximately $44\,N_{\mathrm{gridmax}} - 17\,N_{\mathrm{grids}}$. Since $N_{\mathrm{grids}} \ll N_{\mathrm{gridmax}}$ in practice (typical occupancy is 30–60 per cent), the savings are substantial, reaching ∼176 MB for $N_{\mathrm{gridmax}} = 5\,\mathrm{M}$ at 50 per cent occupancy, a reduction of roughly 60 per cent in neighbour-related memory.

The computational cost of a hash lookup is $\mathcal{O}(1)$ expected time, with worst-case linear probing bounded by the fill fraction. In practice, the precomputed neighbour caches described in Appendix D alleviate any per-lookup overhead in the performance-critical Poisson solver.

The combined memory savings for $N_{\mathrm{gridmax}} = 5\,\mathrm{M}$ exceed 1.2 GB per rank, including `nbor` removal (240 MB), `hilbert_key` elimination (640 MB under `QUADHILBERT`), on-demand `bisec_ind_cell` and `cell_level` (320 MB), and defrag scratch (40 MB), minus hash table overhead (∼50 MB). For standard 64-bit keys the Hilbert savings would be 320 MB.

The freed memory enables deeper AMR hierarchies or larger N_{gridmax} within the same hardware budget.

A diagnostic routine `writemem_minmax` reports the minimum and maximum resident set size across all ranks at each coarse step, providing runtime verification of the memory savings.

APPENDIX D: MULTIGRID SOLVER IMPLEMENTATION DETAILS

D1 Neighbour Grid Precomputation

The Gauss–Seidel (GS) smoother and residual computation both require access to the six Cartesian neighbours of each grid. In the Morton hash table approach (Section 4), each neighbour lookup involves a hash table query. While individual lookups are $\mathcal{O}(1)$, the GS kernel accesses 6 neighbours per grid, 8 cells per grid, and typically 4–5 V-cycle iterations, resulting in hundreds of hash lookups per grid per solve.

We precompute all neighbour grids into a contiguous array before entering the V-cycle iteration loop,

$$\text{nbor_grid_fine}(j,\, i) = \text{morton_nbor_grid}(\text{igrid_amr}(i),\, l,\, j), \tag{D1}$$

where l is the AMR level being solved, igrid_amr(i) is the AMR grid pointer for the i-th active grid at level l, $j = 0, 1, \ldots, 6$ (where $j = 0$ stores the grid's own AMR index), and $i = 1, \ldots, N_{\text{grid}}$. This array is allocated before the iteration loop and deallocated after, so its memory overhead is transient.

D2 Merged Red-Black Exchange and Fused Kernels

We remove the inter-sweep ghost exchange between red and black passes (Red → Black → Exchange instead of Red → Exchange → Black → Exchange), reducing the per-iteration exchange count from 9 to 5. Boundary cells in the black sweep use values from the previous iteration; since the MG solve serves as a preconditioner for the CG outer iteration, this relaxed synchronisation does not affect convergence.

We also remove two unnecessary residual exchanges per iteration, reducing the total from 9 to 5 exchange calls per iteration, a 44 per cent reduction in MG communication volume.

The same optimization is applied to the coarse-level solver (direct solve, pre-smoothing, post-smoothing), where the merged red-black pattern similarly halves the exchange count.

The MG algorithm requires both the residual $r = f - \nabla_h^2 \phi$ and its L^2 norm $\|r\|_2^2$ at specific points in the V-cycle. In the original code, these are computed in separate passes. We add an optional `norm2` argument to `cmp_residual_mg_fine`: when present, the norm is accumulated during the same loop that computes the residual, saving one full grid traversal. Since the subroutine is `external` (not module-contained), callers must include an `interface` block to enable the optional-argument dispatch.

The GS fast-path computation involves a division by $2N_{\text{dim}} = 6$:

$$\phi_{\text{new}} = \frac{\sum_j \phi_j - h^2 f}{2N_{\text{dim}}}. \tag{D2}$$

We replace the division by $2N_{\text{dim}}$ with a multiplication by the precomputed reciprocal $1/(2N_{\text{dim}})$, which is faster on most architectures.

APPENDIX E: NEW NAMELIST PARAMETERS

Table E1 lists the namelist parameters introduced or modified by cuRAMSES.

Table E1. New namelist parameters introduced by cuRAMSES. Default values are shown where applicable.

Parameter	Namelist block	Type	Default	Description
`ordering`	`&RUN_PARAMS`	string	`'hilbert'`	Domain decomposition ordering (`'hilbert'` or `'ksection'`)
`use_fftw`	`&RUN_PARAMS`	logical	`.false.`	Enable FFTW3 direct Poisson solver at base level
`exchange_method`	`&RUN_PARAMS`	string	`'auto'`	MPI exchange backend (`'auto'`, `'p2p'`, `'ksection'`)
`gpu_hydro`	`&RUN_PARAMS`	logical	`.false.`	Enable GPU dispatch for hydrodynamics
`gpu_poisson`	`&RUN_PARAMS`	logical	`.false.`	Enable GPU dispatch for MG Poisson solver
`gpu_fft`	`&RUN_PARAMS`	logical	`.false.`	Enable GPU dispatch for FFT solver
`gpu_sink`	`&RUN_PARAMS`	logical	`.false.`	Enable GPU dispatch for sink particle routines
`memory_balance`	`&RUN_PARAMS`	logical	`.false.`	Enable memory-weighted load balancing
`use_neutrino`	`&RUN_PARAMS`	logical	`.false.`	Enable massive neutrino linear response
`de_perturb`	`&RUN_PARAMS`	logical	`.false.`	Enable dark energy perturbation corrections
`mem_weight_grid`	`&AMR_PARAMS`	integer	auto	Memory cost per grid ($0 =$ auto from `nvar`; $> 0 =$ override)
`mem_weight_part`	`&AMR_PARAMS`	integer	12	Memory cost per particle
`mem_weight_sink`	`&AMR_PARAMS`	integer	500	Computational cost per sink particle
`omega_nu`	`&COSMO_PARAMS`	real	0	Massive neutrino density parameter Ω_ν
`neutrino_table`	`&COSMO_PARAMS`	string	—	Path to neutrino transfer function table